Reproducing Kernel Hilbert Space Choice Model

YIQI YANG, Shanghai Jiao Tong University, China
ZHI WANG, University of Texas at Austin, USA
RUI GAO, University of Texas at Austin, USA
SHUANG LI, Chinese University of Hong Kong, Shenzhen, China

Discrete choice models are essential for understanding human decision-making. Traditional approaches, rooted in Random Utility Models, rely on the assumption of stochastic rationality, limiting their ability to capture context-dependent behavior—where the presence of certain alternatives influences the perceived utility of others. To address these limitations, we propose a novel vector-valued Reproducing Kernel Hilbert Space (RKHS)-based choice model that flexibly captures context effects without imposing strong structural assumptions. Our framework supports both featureless and feature-based settings by introducing set kernels to model choice set interactions and product kernels to incorporate item-specific features. To ensure scalability, we leverage random feature approximations and Neural Tangent Kernels. We demonstrate that our models can be efficiently learned using convex optimization or stochastic gradient descent, and we derive generalization error bounds to guarantee robust performance. Empirical results on synthetic and real-world datasets show that our approach outperforms traditional discrete choice models in both predictive accuracy and generalization.

1 Introduction

Discrete choice models are foundational tools for understanding and predicting human decision-making across a wide range of fields. From economics [McFadden, 2001] and transportation [Brownstone, 2000, McFadden et al., 1977] to marketing [Chintagunta and Nair, 2011, McFadden, 1986] and operations research [Talluri and Van Ryzin, 2004, Wang, 2021], these models describe how individuals select among alternatives based on their preferences and the attributes of the available options. The heart of classical choice theory lies in the framework of Random Utility Models (RUMs) [Marschak, 1959, Thurstone, 1927], which posits that individuals make rational decisions by maximizing a utility function composed of observed attributes and random noise. The Multinomial Logit (MNL) model [Luce, 1959, McFadden, 1974a] emerged as a cornerstone of this framework, followed by extensions such as Nested MNL [McFadden, 1974b, Williams, 1977] and Mixed MNL [McFadden and Train, 2000] introducing hierarchical structures and random coefficients to enhance modeling flexibility. Recent advances have further expanded the field, with modern models like non-parametric [Farias et al., 2013], Exponomial [Alptekinoğlu and Semple, 2016], Markov chain [Blanchet et al., 2016], RUM neural net [Aouad and Désir, 2023] choice models, enabling the capture of increasingly complex choice behaviors.

Despite their widespread adoption, traditional discrete choice models rely on the assumption of stochastic rationality [Gallego and Iravani, 2024]. While this assumption has profound theoretical implications and simplifies estimation, it often falls short in capturing the rich complexity of real-world decision-making [Jagabathula and Rusmevichientong, 2018], where choices are often influenced by contextual factors and psychological biases. Empirical studies in marketing and cognitive science have consistently observed systematic deviations from rational behavior, known as *context effects* [Tversky and Simonson, 1993], where the presence of certain alternatives can significantly influence the perceived utility of others. By failing to account for these behavioral influences, classical choice models risk misestimated choice probabilities and suboptimal predictions.

Modeling context-dependent choice behavior presents significant challenges. A generic choice model with context-dependent utilities must account for all interactions among items in the choice set, which can grow exponentially with the number of items. For instance, in a choice set of size

J, there are 2^J possible subsets, each of which may influence the utility of the items within it. This combinatorial explosion of variables creates a threefold challenge: balancing expressiveness—the ability to capture complex context effects—generalization—the ability to avoid overfitting to limited training data—and computational efficiency—the ability to scale to large choice sets and datasets. Existing approaches often rely on predefined structural assumptions or heuristic approximations, which might not fully capture the nuanced ways in which context shapes decisions. As a result, there is a critical need for models that can simultaneously achieve high expressiveness, robust generalization, and computational tractability.

In this paper, we propose a novel vector-valued Reproducing Kernel Hilbert Space (RKHS)-based choice model that addresses these challenges by combining the flexibility of nonparametric methods with the theoretical rigor of kernel-based learning. Our framework is designed to model both featureless choice settings, where only the choice set itself is observed, and settings with item-specific features, where additional contextual information is available. By leveraging the rich mathematical structure of RKHS, we provide a principled, data-driven approach to learning context-dependent utility functions that can capture high-order interactions between items while avoiding overfitting.

1.1 Main Contributions

Our work makes the following key contributions.

(i) We introduce a rigorous RKHS framework for discrete choice modeling (Section 2). For the featureless case, we define a vector-valued RKHS over set functions and introduce a masking mechanism to handle variable-sized choice sets. For settings with features, we propose a product kernel that combines a set kernel with a feature kernel, enabling the model to incorporate both choice set structure and item-specific attributes. In both cases, we establish the validity of these kernels.

(ii) We explore a variety of kernel choices to model context effects and feature interactions (Section 3). For set kernels, we discuss designs that capture context-dependent behaviors, such as the Halo effect [Nisbett and Wilson, 1977]. For feature kernels, in addition to classic choices like Gaussian and Matérn kernels, we propose efficient implementations using random feature approximations, enabling scalability to large datasets. To model high-dimensional, nonlinear relationships in choice behavior, we leverage Neural Tangent Kernel theory as a powerful analytical tool for multi-head attention-based choice models. This approach allows the model to adaptively learn feature interactions and context effects, effectively bridging the gap between kernel methods and deep learning.

(iii) We address the statistical estimation problem for learning the proposed models (Section 4). For fixed kernels, we leverage the Representer Theorem to reformulate the estimation problem as a convex optimization task, reducing the infinite-dimensional problem to a tractable finite-dimensional one. In the case of random kernel approximations, we demonstrate how they enhance computational efficiency. For neural tangent kernels, we show that stochastic gradient descent can efficiently converge to a nearly optimal solution. Additionally, we derive generalization error bounds for these models, ensuring that the learned utility functions perform well on unseen data without suffering from the curse of dimensionality.

(iv) We perform numerical experiments on both synthetic and real-world datasets to validate the effectiveness of our RKHS-based choice model (Section 5). The empirical results show that

our model surpasses traditional approaches in both predictive accuracy and generalization capability.

In summary, our RKHS-based framework provides a scalable, data-driven approach to capturing intricate context effects in discrete choice modeling, offering greater flexibility, expressiveness, and generalization.

1.2 Related Literature

Empirical studies in cognitive science and marketing have identified various systematic deviations from the traditional rational choice models, highlighting the role of context effects in decision-making. For example, the similarity effect [Tversky, 1972] demonstrates that adding a new option disproportionately reduces the likelihood of selecting similar existing options. The attraction (decoy) effect [Huber et al., 1982] reveals that introducing a dominated option can increase the likelihood of selecting a target alternative. Similarly, the compromise effect [Simonson, 1989] shows that individuals tend to prefer middle-ground options when extreme alternatives are present. In addition to these context effects, cognitive biases such as loss aversion and the framing effect [Kahneman and Tversky, 1979, Tversky and Kahneman, 1991] further illustrate the limitations of classical models.

To address these limitations, researchers have developed a range of context-dependent choice models that relax the strict assumptions of RUM. One such approach is Context MNL [Ko and Li, 2024, Maragheh et al., 2018, Seshadri et al., 2019, Yousefi Maragheh et al., 2020], which explicitly models linear context effects by allowing the utility of each alternative to depend on the presence or absence of other items in the choice set. However, these works do not consider item-specific features, limiting their applicability in more complex decision environments. Moreover, Context MNL can be seen as a special case of our set kernel (Section 3.1), as we extend beyond linear effects to capture higher-order nonlinear interactions, providing a more expressive and flexible representation of choice behavior.

More recently, deep learning approaches have been applied to choice modeling. [Wang et al., 2023a] leverages ResNet [He et al., 2016] and [Pfannschmidt et al., 2022, Rosenfeld et al., 2020] utilize Deep Set [Zaheer et al., 2017] to account for the assortment and set-dependent effect, whereas our work employs attention-based neural nets to capture high-order context effects, similar to [Ko and Li, 2024, Wang et al., 2023b]. While these models mostly focus on single-head attention mechanisms in theoretical development, our work emphasizes the benefits of wide multi-head attention, which enables a richer representation and facilitates connections to neural tangent kernel (NTK) theory for better theoretical understanding and efficiency.

Another prominent stream of research focuses on consider-then-choose models [Akchen and Mitrofanov, 2024, Aouad et al., 2021, Brady and Rehbeck, 2016, Gallego and Iravani, 2024, Wang, 2022]. These models assume that individuals first screen products to determine which options to consider before ranking and making a final choice. This two-stage decision-making process aligns well with real-world scenarios where consumers use heuristics to narrow down choices before making a final selection.

Finally, we note that RKHS has been explored in the context of choice modeling, mostly focusing on fixed choice set. Within the fixed choice set paradigm, context-independent approaches have been extensively explored, with studies such as [Martín-Baos et al., 2021a,b] revisiting traditional Random Utility Models (RUM) and [Espinosa-Aranda et al., 2018] introducing constrained RUM formulations. Additionally, [Singh and Vijaykumar, 2023] proposed a rank kernel approach, potentially subsumed under feature kernel methods that incorporate ranking positions as features. In contrast, context-dependent models within fixed choice sets, such as those by [Chang et al.,

2025], adopt scalar-valued, classification-based techniques like kernel ridge regression, emphasizing consumer-specific preferences over item-specific attributes [Martín-Baos et al., 2025]. Dynamic choice models, as explored by [Li et al., 2023] in the context of reinforcement learning from human feedback, further extend this framework by allowing preferences to evolve over time, leveraging RKHS modeling for reward functions. On the other hand, the work [Espinosa-Aranda et al., 2015] allows for varied choice set and proposes scalar-valued, context-dependent approaches under a constrained nested logit framework, specifically for railway scheduling applications. In comparison to these scalar-valued RKHS models, our work considers a vector-valued RKHS choice model that can adapt to choice sets with varied cardinality and capture more complex item-wise interaction effects within the universal logit structure.

2 RKHS Choice Model

In this section, we introduce our RKHS-based framework for discrete choice modeling. We begin by a brief description of the general discrete choice model, followed by a detailed description of our RKHS-based framework, which includes a basic model without features and an extended model with features.

2.1 Model Setup

Let S denote a choice set, and let $p_j(S)$ represent the probability of choosing item j from the choice set S. Consider a generic choice model of the form

$$p_j(S) = \frac{\exp(u_j(S))}{\sum_{j' \in S} \exp(u_{j'}(S))}, \quad j \in S,$$

where the choice probability $p_j(S)$ is determined by a menu-dependent utility function $u_j(S)$ that depends on the entire choice set S.

The equation above represents a general form of a discrete choice model, capable of capturing any choice behaviors [Manski, 1977]. However, the primary challenge lies in specifying the utility function $\boldsymbol{v}$. Specifically, let $\mathcal{S}$ denote the universe of possible items. Then the utility function $\boldsymbol{u}$ is a set function defined for every possible choice set. Since the number of possible choice sets grows exponentially with the size of $\mathcal{S}$, whereas empirical observations are typically limited to a small subset of these sets, specifying $\boldsymbol{u}$ becomes computationally and statistically challenging.

To facilitates the discussion, we identify the choice set S with its indicator vector $\boldsymbol{e}_S \in \{0,1\}^J$, where J is the number of items in the universe, and the jth component of $\boldsymbol{e}_S$ is 1 if $j \in S$ and 0 otherwise. We denote by $\boldsymbol{e}_j$ the indicator vector of item j, which is a vector of length J with a 1 in the jth position and 0 elsewhere.

2.2 Model Without Features

In our RKHS-based framework, we assume that the utility function $\boldsymbol{u}$ is a member of a vector-valued RKHS defined on the space of sets. This assumption allows us to leverage the properties of RKHS to model the utility function in a flexible and expressive manner.

We start by defining a vector-valued RKHS on the space of sets. Let $(\mathcal{H}, \langle\cdot,\cdot\rangle\|_{\mathcal{H}})$ be a $\mathbb{R}^J$-valued Hilbert space, equipped with the inner product $\langle\cdot,\cdot\rangle_{\mathcal{H}}$ and the induced norm $\|\cdot\|_{\mathcal{H}}$. A function $\boldsymbol{K} : \mathcal{S} \times \mathcal{S} \to \mathbb{R}^{J\times J}$ is called a *kernel* if it satisfies *(a)* symmetry:

$$\boldsymbol{K}(S,S') = \boldsymbol{K}(S,S')^\top, \quad \forall S,S' \in \mathcal{S},$$

and *(b)* positive definiteness:

$$\sum_{i=1}^{n}\sum_{k=1}^{n} \boldsymbol{\alpha}_i^\top \boldsymbol{K}(S_i, S_k)\boldsymbol{\alpha}_k \geq 0, \quad \forall \{S_i\}_{i=1}^n \subset \mathcal{S}, \ \forall \boldsymbol{\alpha}_1, \ldots, \boldsymbol{v}_n \in \mathbb{R}^J.$$

A vector-valued RKHS is characterized by a kernel $\boldsymbol{K} : \mathcal{S} \times \mathcal{S} \to \mathbb{R}^{J\times J}$

$$\boldsymbol{K} = \begin{bmatrix} \mathsf{k}_{11} & \mathsf{k}_{12} & \ldots & \mathsf{k}_{1J} \\ \mathsf{k}_{21} & \mathsf{k}_{22} & \ldots & \mathsf{k}_{2J} \\ \vdots & \vdots & \ddots & \vdots \\ \mathsf{k}_{J1} & \mathsf{k}_{J2} & \ldots & \mathsf{k}_{JJ} \end{bmatrix}.$$

which maps pairs of choice sets to a matrix of real values. The kernel $\boldsymbol{K}$ encodes the similarity between choice sets, and in particular, the entry $\boldsymbol{K}_{jk}(S, S')$ quantifies the similarity between the jth and kth items in the context of choice sets S and S'.

The Hilbert space $\mathcal{H}$ is an RKHS if it satisfies the *reproducing property*, which states that for every choice set $S \subset \mathcal{S}$ and every function $\boldsymbol{u} \in \mathcal{H}$, the the evaluation of $\boldsymbol{u}$ at S can be expressed as an inner product in $\mathcal{H}$:

$$u_j(S) = \langle \boldsymbol{u}, \boldsymbol{K}(\cdot, S)\boldsymbol{e}_j\rangle_{\mathcal{H}}, \quad j \in S,$$

where $\boldsymbol{K}(\cdot, S)$ is the kernel function centered at S. This property allows us to represent the utility function in terms of the kernel, providing a computationally tractable way to model complex dependencies between choice sets and their associated utilities.

Moreover, the kernel $\boldsymbol{K}$ can be designed to capture specific structures or relationships in the choice sets, such as overlaps in items, contextual similarities, or hierarchical dependencies. By choosing an appropriate kernel, we can tailor the RKHS framework to reflect domain-specific knowledge or behavioral assumptions, ensuring that the utility function $\boldsymbol{u}$ is both interpretable and predictive. We will discuss potential choices of kernels in Section 3.

The space $\mathcal{H}$ can be represented as the closure of the set of functions of the form

$$\boldsymbol{u} = \sum_{i=1}^{\infty} \boldsymbol{K}(\cdot, S_i)\boldsymbol{\alpha}_i,$$

where $\{S_i\}_{i=1}^{\infty}$ is a sequence of choice sets, $\{\boldsymbol{\alpha}_i\}_{i=1}^{\infty}$ are coefficient vectors in $\mathbb{R}^J$, and $\boldsymbol{K}(\cdot, S_i)$ denotes the kernel function centered at S_i. The infinite sum above is assumed to converge in the norm of $\mathcal{H}$, ensuring that $\boldsymbol{u}$ is a well-defined function in the RKHS. For functions $\boldsymbol{u}, \boldsymbol{v} \in \mathcal{H}$ of the above form, their inner product can be expressed as

$$\langle \boldsymbol{u}, \boldsymbol{v}\rangle_{\mathcal{H}} = \sum_{i=1}^{\infty}\sum_{j=1}^{\infty} \boldsymbol{\alpha}_i^\top \boldsymbol{K}(S_i, S_j)\boldsymbol{\beta}_j, \tag{1}$$

where $\boldsymbol{v} = \sum_{j=1}^{\infty} \boldsymbol{K}(\cdot, S_j)\boldsymbol{\beta}_j$ is another function in $\mathcal{H}$. This form will play a crucial role in learning the utility function, as will be discussed in Section 4.

2.2.1 Kernel Masking

To account for the fact that the utility of an item is not defined when it is not included in the choice set, we introduce a *masking* mechanism. This mechanism ensures that $u_j(S) = 0$ when an item j is not in the choice set S. Specifically, given a kernel $\boldsymbol{K}$, we define a masked function $\boldsymbol{K}^{\mathcal{S}}$ as

$$\boldsymbol{K}^{\mathcal{S}} = [\mathsf{k}_{jk}^{\mathcal{S}}]_{j,k=1,\ldots,J}, \text{ with } \mathsf{k}_{jk}^{\mathcal{S}}(S, S') = \mathbb{1}(j \in S) \cdot \mathbb{1}(k \in S') \cdot \mathsf{k}_{jk}(S, S'), \tag{2}$$

where $\mathbb{1}(\cdot)$ is the indicator function. In the following proposition, we show that the masked function $\boldsymbol{K}^{\mathcal{S}}$ is indeed a kernel and derive the corresponding RKHS norm.

Proposition 1 (Set Kernel). *The masked function $\boldsymbol{K}^{\mathcal{S}}$ defined in* (2) *is a kernel. Moreover, for a vector-valued utility function $\boldsymbol{u} \in \mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}$ of the form $\boldsymbol{u} = \sum_{i=1}^{n} \boldsymbol{K}^{\mathcal{S}}(\cdot, S_i)\boldsymbol{\alpha}_i$, its RKHS norm satisfies*

$$\|\boldsymbol{u}\|^2_{\mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}} = \sum_{i=1}^{n} \sum_{j \in S_i} u_j(S_i)\alpha_{ij}.$$

This result demonstrates that, under our construction of the masked kernel $\boldsymbol{K}^{\mathcal{S}}$, the corresponding RKHS norm of a utility function $\boldsymbol{u}$ is independent of the utilities $\{u_j(S_i)\}_{j \notin S_i}$ for items that are not in the choice set S_i, $i = 1, \ldots, n$. This property aligns with our model setup in Section 2.1, where the choice probability $p_j(S)$ is defined soley based on the utilities of items within the choice set S.

2.3 Model With Features

While the basic model provides a flexible framework for modeling choice behavior through menu-dependent utilities, it does not explicitly account for the attributes or features of the items in the choice set. In many applications, however, the utility of an item is not only influenced by the choice set itself but also by the characteristics of the items, such as price, quality, or other contextual factors. To capture these effects, we extend the model to incorporate feature information.

To unify the notation, we assume that the maximum number of items in any choice set S is finite, denoted as J. Let d be the feature dimension of each item. We denote by $\boldsymbol{X}_S$ the $|S| \times d$-dimensional feature matrix for the items in the choice set S, where each row corresponds to the feature vector of an item in S. The utility function $\boldsymbol{u}(S, \boldsymbol{X}_S)$ is now a function of both the choice set S and its associated features $\boldsymbol{X}_S$. By padding zeros to the feature matrix, we can assume that the feature matrix $\boldsymbol{X}_S$ is of size $J \times d$, where the rows corresponding to items not in S are padded with zeros. Moreover, we can always add a row of ones to the feature matrix $\boldsymbol{X}_S$ to account for the intercept term. For convenience, we denote by $\boldsymbol{X}_S$ the feature matrix with the intercept term included whenever it is needed.

Under the above setup, we extend the definition of the kernel to depend on both the choice sets and the features:

$$\bar{\boldsymbol{K}} : (\{0,1\}^J \times \mathbb{R}^{J \times d}) \times (\{0,1\}^J \times \mathbb{R}^{J \times d}) \to \mathbb{R}^{J \times J},$$

where the choice set S is identified with its indicator vector $\boldsymbol{e}_S \in \{0,1\}^J$, and $\mathbb{R}^{|S| \times d}$ is the space of feature matrices for the items in S. The kernel $\boldsymbol{K}$ can be designed to incorporate both the choice set structure and the feature information. Specifically, we consider a product kernel of the following form that combines a set kernel $\boldsymbol{K}^{\mathcal{S}}$ and a feature kernel $\boldsymbol{K}^{\mathcal{X}}$:

$$\bar{\boldsymbol{K}}\big((S, \boldsymbol{X}_S), (S', \boldsymbol{X}_{S'})\big) := \boldsymbol{K}^{\mathcal{S}}(S, S') \cdot \boldsymbol{K}^{\mathcal{X}}(\boldsymbol{X}_S, \boldsymbol{X}_{S'}).$$

The set kernel $\boldsymbol{K}^{\mathcal{S}}$ can be chosen as the masked kernel defined in (2), while the feature kernel $\boldsymbol{K}^{\mathcal{X}}$ can be defined using standard kernels, such as the Gaussian kernel, Matérn kernel, as detailed in the next Section.

3 Choices of Kernels

This section examines three types of kernels. Sections 3.1-3.2 focus on traditional fixed kernels for sets and features. Section 3.3 discusses a random feature method for kernel approximation. Finally, Section 3.4 explores a data-driven, adaptive kernel that leverages neural networks to model more complex and data-dependent structures.

3.1 Set Kernels

Identifying S with its indicator vector $\boldsymbol{e}_S$, one can adapt common kernels for real-valued functions to set functions. In particular, taking $\boldsymbol{e}_S, \boldsymbol{e}_{S'}$ as the inputs, one common structure to specify the unmasked matrix-valued set kernel is by a scalar kernel $\mathsf{k}(\boldsymbol{e}_S, \boldsymbol{e}_{S'})$ multiplied by a positive semi-definite matrix A, given by

$$\boldsymbol{K}(S, S') = \mathsf{k}(S, S') \cdot A. \tag{3}$$

When A is set to an identity matrix, the above kernel is reduced to the scalar-valued RKHS model in Ángel Martín-Baos et al. [2024] that is inspired from multi-class classification. Yet, they do not consider masking, as the choice sets are all identical in their setting.

3.1.1 Halo's Effect

Among many choices of the scalar kernel in (3.1), here we examine the polynomial kernel for its close connection to the Halo effect, offering interpretable insights.

Consider a p-order Halo effect model that accounts for interactions with up to p other products, which generalized the pair-wise interaction model discussed in Batsell and Polking [1985], Seshadri et al. [2019]. For any choice set $S \subset \mathcal{S}$, we have

$$u_j(S) = v_j + \sum_{k \in S \setminus \{j\}} v_j^k + \sum_{k_1 < k_2 : k_1, k_2 \in S \setminus \{j\}} v_j^{k_1, k_2} + \cdots + \sum_{k_1 < \cdots < k_p : k_1, \ldots, k_p \in S \setminus \{j\}} v_j^{k_1, \ldots, k_p}, \quad \forall j \in S, \tag{4}$$

where v_j represents the base utility of item j, v_j^k captures the first-order interaction effect on item j when item k is in the choice set, $v_j^{k_1,k_2}$ captures the second-order interaction effect on item j when items k_1 and k_2 are in the choice set, and so on.

The following proposition shows that our RKHS model with a p-order polynomial kernel can be used to capture up to p-th order interaction effects.

Proposition 2. *The model* (4) *can be represented by an RKHS model* (2) *with a p-order polynomial kernel, specified by*

$$\mathsf{k}^{\mathcal{S}}_{jk}(S, S') = \mathbb{1}(j \in S) \cdot \mathbb{1}(k \in S) \cdot (\boldsymbol{e}_S^\top \boldsymbol{e}_{S'})^p.$$

As can be seen from the proof, the coefficients in (4) can be computed using the feature map associated with the polynomial kernel. In Section 5.1, we perform synthetic experiments to compare our RKHS model with a polynomial kernel with a method that directly fits the polynomial coefficients in (4).

3.2 Feature Kernels

In terms of modeling with features, we consider the feature kernel $\boldsymbol{K}^{\mathcal{X}}(\boldsymbol{X}_S, \boldsymbol{X}_{S'}) \in \mathbb{R}^{J \times J}$ to be a similarity matrix, where its (i, j)-th entry represents the similarity between the i-th row of $\boldsymbol{X}_S$ and the j-th row of $\boldsymbol{X}_{S'}$. The Gaussian kernel and the Matérn kernel are widely used because they are universal kernels that enable RKHS to approximate any continuous function arbitrarily well. For example, each entry of the similarity matrix of Gaussian kernel is given by

$$\left[\boldsymbol{K}^{\mathcal{X}}(\boldsymbol{X}_S, \boldsymbol{X}_{S'})\right]_{ij} = \sigma^2 \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}'_j\|_2^2}{2\ell^2}\right),$$

where $\boldsymbol{x}_i, \boldsymbol{x}'_j \in \mathbb{R}^d$ is the i-th and j-th rows of $\boldsymbol{X}_S$ and $\boldsymbol{X}_{S'}$, respectively. ℓ is a tuning parameter which we set to the median of $\{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2\}$ for the dataset. Similarly, each entry of Matérn kernel can be set to

$$\left[\boldsymbol{K}^{\mathcal{X}}(\boldsymbol{X}_S, \boldsymbol{X}_{S'})\right]_{ij} = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{\|\boldsymbol{x}_i - \boldsymbol{x}'_j\|_2}{\ell}\right)^{\nu} B_\nu\left(\sqrt{2\nu} \frac{\|\boldsymbol{x}_i - \boldsymbol{x}'_j\|_2}{\ell}\right),$$

which generalizes Gaussian kernel. Notably, this similarity score assumes a row-wise comparison, which is effective when items at the same row index belong to a comparable category. For example, in transportation preference modeling, each row represents a specific type of mobility tool (e.g., cars) with varying feature values (e.g., travel time and price).

For other stationary kernels, we can also specify the corresponding matrix-valued feature kernels. The p-order polynomial scalar kernel is set to be

$$\frac{(\boldsymbol{x}_i^\top \boldsymbol{x}_j' + c)^p}{(\max_{i,j} \boldsymbol{x}_i^\top \boldsymbol{x}_j' + c)^p}.$$

where the scaling helps alleviate kernel bias towards inputs with larger magnitudes. Moreover, the featureless set kernel can be obtained by simply setting $\boldsymbol{X}_S = \boldsymbol{e}_S$.

3.3 Random Feature Methods for Kernel Approximation

Optimizing the problem in Theorem 1 with the fixed (stationary) kernels requires pre-computing and storing all the kernel values. This presents a substantial memory challenge making large-scale applications infeasible. One practical approach to tackle this challenge is the *random feature* method [Rahimi and Recht, 2007], which efficiently approximates stationary kernels by mapping data into a lower-dimensional randomized feature space. It aims to avoid the expensive operations associated with the high-dimensional kernel matrices while maintaining good approximation quality.

Let $\boldsymbol{\phi} : (\{0,1\}^J \times \mathbb{R}^{J\times d}) \times \mathcal{W} \to \mathbb{R}^J$ be a feature map parameterized by $w \in \mathcal{W}$ and $Q(w)$ be a probability measure on a feature space $\mathcal{W}$. We define the following kernel via random feature:

$$\boldsymbol{K}_Q\big((S, \boldsymbol{X}_S), (S', \boldsymbol{X}_{S'})\big) = \mathbb{E}_Q\Big[\boldsymbol{\phi}(S, \boldsymbol{X}_S; w)\boldsymbol{\phi}\big(S', \boldsymbol{X}_{S'}; w\big)^\top\Big]. \tag{5}$$

Note that it is difficult to optimize the distribution Q directly. To make it tractable, we discretize the distribution by N_w points, each represented by a sample w_m (sampled from some pre-determined distribution) with a corresponding frequency q_m. Then (5) is approximated by

$$\sum_{m=1}^{N_w} q_m \boldsymbol{\phi}(S, X_S; w_m)\boldsymbol{\phi}(S', X_{S'}; w_m)^\top,$$

where the frequencies $\{q_m\}_m$ can be obtained using optimization, which will be introduced in Section 4.2.

As an illustration, given $\{q_m\}_m$, let us approximate the matrix-valued Gaussian kernels using the above discrete representation for both the featureless and feature-based settings. In the featureless model, for each $m \in [N_w]$, we independently sample J pairs $\{(\boldsymbol{\omega}_m^j, b_m^j)\}_{j=1}^J$ where $\boldsymbol{\omega}_m^j \sim \mathcal{N}\left(0, \frac{\boldsymbol{I}}{\ell^2}\right)$ and $b_m^j \sim \mathcal{U}(0, 2\pi)$. The corresponding feature map with respect to $w_m := [\{\boldsymbol{\omega}_1, \cdots, \boldsymbol{\omega}_J\}, \{b_1, \cdots, b_J\}]$ is given by

$$\boldsymbol{\phi}(S, w_m) = \sqrt{2}\sigma\ \left[\cos(\boldsymbol{\omega}_1^\top \boldsymbol{e}_S + b_1), \cdots, \cos(\boldsymbol{\omega}_J^\top \boldsymbol{e}_S + b_J)\right]^\top.$$

Meanwhile, the feature-based feature map takes in $\boldsymbol{X}_S$ and a parameter shared across rows. For each $m \in [N_w]$, we sample a single vector $\boldsymbol{\omega}_m \sim \mathcal{N}(0, \boldsymbol{I}/\ell_2)$ and offset $b_m \sim \mathcal{U}(0, 2\pi)$. The respective feature map associated with $w_m = [\boldsymbol{\omega}_m, b_m]$ is defined as

$$\boldsymbol{\phi}(\boldsymbol{X}_S, w_m) = \sqrt{2}\sigma\left[\cos(\boldsymbol{\omega}^\top \boldsymbol{x}_{S,1} + b), \cdots, \cos(\boldsymbol{\omega}^\top \boldsymbol{x}_{S,J} + b)\right]^\top.$$

The above random feature model uses a feature map parameterized by $\boldsymbol{\omega}$ and b, reducing the computational and memory complexity from $O(N^2J^2)$ to $O(N_wNJ)$. This avoids the requirement

of mixed-precision training for fixed kernels with large-scale datasets. Nonetheless, this efficiency gain comes at the cost of approximation error, which may downplay the performance compared to traditional kernels.

3.4 Neural Tangent Kernel

While the kernels discussed earlier provide a principled way to model similarities between choice sets and their features, they are typically predefined and fixed before training. This limits their ability to capture complex, data-dependent patterns in real-world choice behavior. To address this, we introduce the *Neural Tangent Kernel* (NTK) as an adaptive kernel that learns from data and captures intricate feature interactions. Defined implicitly by a neural network's architecture, NTKs evolve during training, leveraging the expressive power of neural networks to adapt to input structure. This flexibility makes them well-suited for modeling high-dimensional, nonlinear relationships in choice behavior while preserving the theoretical guarantees of kernel methods.

3.4.1 Two-Layer Neural Network with an Attention Layer

Let $\boldsymbol{x}_j$ be the feature vector of item j in the choice set S, corresponding to the j-th row of the feature matrix X_S. To capture pairwise similarities between items and context-aware representations, we employ a *self-attention mechanism* [Ko and Li, 2024, Vaswani, 2017, Wang et al., 2023b]. This mechanism allows the model to weigh the importance of different items in the choice set when making predictions. Specifically, the first layer of the neural network is a self-attention layer that computes a context-dependent representation for each item in the choice set. The second layer is a linear layer that combines these representations to produce the final utility values.

To define the attention block in the first layer, we first define the attention matrix A as

$$A = [a_{jk}]_{j,k\in S}, \text{ with } a_{jk} = \frac{(\Theta_1 \boldsymbol{x}_j)^\top (\Theta_2 \boldsymbol{x}_k)}{\sqrt{d_0}}, \tag{6}$$

where $\Theta_1, \Theta_2 \in \mathbb{R}^{d_0\times d}$ are learnable weight matrices that transform the feature vectors to a d_0-dimensional space. The attention score a_{jk} is the inner product of the transformed feature vectors normalized by $\sqrt{d_0}$, and measures the interaction between items j and k in the choice set S. For each item $j \in S$, we compute the attention weights $\boldsymbol{\omega}_j = [\omega_{jk}]_{k\in S} \in \mathbb{R}^{|S|}$ using a nonlinear mapping $\sigma : \mathbb{R}^{|S|} \to \mathbb{R}^{|S|}$ (e.g., softmax function)

$$\boldsymbol{\omega}_j = \sigma(\boldsymbol{a}_j) = \sigma\Big(\frac{(\Theta_1 \boldsymbol{x}_j)^\top (\Theta_2 \boldsymbol{X}_S^\top)}{\sqrt{d_0}}\Big).$$

These weights $\{\omega_{jk}\}$ determine the relative impact of item k on the representation of item j, thus capturing the context effect. The context-dependent representation $\boldsymbol{z} = [z_j]_{j\in S}$ is computed as

$$z_j = \boldsymbol{\omega}_j \boldsymbol{X}_S \Theta_3^\top = \sum_{k\in S} \omega_{jk} \Theta_3 \boldsymbol{x}_k, \quad j \in S,$$

where $\Theta_3 \in \mathbb{R}^{d_1\times d}$ is another learnable weight matrix that transforms the feature vectors to a d_1-dimensional space. Let $\boldsymbol{z} \in \mathbb{R}^{J\times d_1}$ be the concatenation of all z_j for $j \in S$ padding zeros if necessary. We denote the output of the attention block as

$$\phi(\boldsymbol{X}_S; \Theta) = \boldsymbol{z} = \sigma\Big(\frac{(\Theta_1 \boldsymbol{X}_S^\top)^\top (\Theta_2 \boldsymbol{X}_S^\top)}{\sqrt{d_0}}\Big) \boldsymbol{X}_S \Theta_3^\top. \tag{7}$$

The attention-based feature map $\phi_j(\boldsymbol{X}_S; \Theta)$ captures the influence of all items in the choice set on item j, $j \in S$. The parameters Θ_1, Θ_2 parameterize the context effect, whereas Θ_3 parameterizes the feature transformation.

If we think of the attention score matrix (6) as a kernel matrix, it is natural to use multiple attention matrices, so as to capture different notions of similarity. This is called the *multi-head attention*. Specifically, we use H different attention maps $\phi(\boldsymbol{X}_S;\Theta^h)$, each with its own set of parameters $\Theta^h = (\Theta_1^h, \Theta_2^h, \Theta_3^h)$, $h = 1, \cdots, H$.

In the second layer, we use a linear layer to aggregate the H context-dependent representations from the first layer to produce the final utility values $\boldsymbol{v} \in \mathbb{R}^J$:

$$\boldsymbol{v}(\boldsymbol{X}_S;\boldsymbol{\Theta},\boldsymbol{w}) = \frac{1}{\sqrt{H}}\sum_{h=1}^{H}\phi(\boldsymbol{X}_S;\Theta^h)w^h, \tag{8}$$

where $\boldsymbol{w} = [w^h]_{h=1}^H$, $\boldsymbol{\Theta} = [\Theta^h]_{h=1}^H$, $w^h \in \mathbb{R}^{d_1}$ is the weight vector for the h-th attention feature map in the second layer, and the scaling factor $\frac{1}{\sqrt{H}}$ will be useful to provide a kernel representation of the neural network.

3.4.2 Multi-layer Attention (DeepNTK) To increase model expressivenss, we consider L layers model:

$$\boldsymbol{v}_1 := \frac{1}{\sqrt{H}}\sum_{h=1}^{H}\phi(\boldsymbol{X}_S;\Theta^h)W^{h,1} \in \mathbb{R}^{J\times d_2},$$

$$\boldsymbol{v}_l := \frac{1}{\sqrt{H}}\sum_{h=1}^{H}\phi(\sigma(\boldsymbol{v}_{l-1});\tilde{\Theta}^{h,l})W^{h,l} \in \mathbb{R}^{J\times d_2}, \quad \forall l = 2,\ldots,L-1,$$

$$\boldsymbol{v}_L := \frac{1}{\sqrt{H}}\sum_{h=1}^{H}\phi(\sigma(\boldsymbol{v}_{L-1});\tilde{\Theta}^{h,L})w^h \in \mathbb{R}^{J},$$

where for every $h = 1,\ldots,H$: for $l = 2,\ldots,L$, $\tilde{\Theta}_1^{h,l}, \tilde{\Theta}_2^{h,l} \in \mathbb{R}^{d_0\times d_2}$, $\tilde{\Theta}_3^{h,l} \in \mathbb{R}^{d_1\times d_2}$, $\tilde{\Theta}^{h,l} = [\tilde{\Theta}_1^{h,l}, \tilde{\Theta}_2^{h,l}, \tilde{\Theta}_3^{h,l}]$; for $l = 1,\ldots,L-1$, $W^{h,l} \in \mathbb{R}^{d_1\times d_2}$; and $\Theta_1^1, \Theta_2^1 \in \mathbb{R}^{d_0\times d}$, $\Theta_3^1 \in \mathbb{R}^{d_1\times d}$, $\Theta^1 = [\Theta_1^1, \Theta_2^1, \Theta_3^1]$, $w^h \in \mathbb{R}^{d_1}$. Note that $\boldsymbol{v}_l$ denotes the output of the l-th layer and $\boldsymbol{v}_L$ is the output of the deep NTK.

3.4.3 Neural Tangent Kernel Representation

The Neural Tangent Kernel (NTK) bridges the expressive power of neural networks with the theoretical guarantees of kernel methods. To derive the NTK, consider the linearization of the neural network output $\boldsymbol{v}$ in (8) around a random initialization $(\tilde{\boldsymbol{\Theta}},\tilde{\boldsymbol{w}}) \sim \rho$:

$$\begin{aligned}
&\boldsymbol{v}(\boldsymbol{X}_S;\boldsymbol{\Theta},\boldsymbol{w}) - \boldsymbol{v}(\boldsymbol{X}_S;\tilde{\boldsymbol{\Theta}},\tilde{\boldsymbol{w}}) \\
&\qquad \approx \nabla_{\boldsymbol{w}}\boldsymbol{v}(\boldsymbol{X}_S;\tilde{\boldsymbol{\Theta}},\tilde{\boldsymbol{w}})(\boldsymbol{w}-\tilde{\boldsymbol{w}}) + \nabla_{\boldsymbol{\Theta}}\boldsymbol{v}(\boldsymbol{X}_S;\tilde{\boldsymbol{\Theta}},\tilde{\boldsymbol{w}})(\boldsymbol{\Theta}-\tilde{\boldsymbol{\Theta}}) \\
&\qquad = \frac{1}{\sqrt{H}}\sum_{h=1}^{H}\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)(w^h-\tilde{w}^h) + \frac{1}{\sqrt{H}}\sum_{h=1}^{H}(\Theta^h-\tilde{\Theta}^h)^\top\nabla_\Theta\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)\tilde{w}^h.
\end{aligned}$$

According to the NTK theory [Jacot et al., 2018], under the scaling regime $1/\sqrt{H}$, such a linear approximation remains accurate as $H \to \infty$ when the neural network is trained using gradient descent with a small learning rate. This linear approximation exploits two random features $\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)$ and $\nabla_\Theta\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)\tilde{w}^h$. with the corresponding kernels

$$\frac{1}{H}\sum_{h=1}^{H}\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)\phi(\boldsymbol{X}'_{S'};\tilde{\Theta}^h)^\top$$

and

$$\frac{1}{H}\sum_{h=1}^{H}\nabla_\Theta\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)\tilde{w}^h\big(\nabla_\Theta\phi(\boldsymbol{X}_S;\tilde{\Theta}^h)\tilde{w}^h\big)^\top.$$

In the infinite-width limit $H \to \infty$, their sum converges to a deterministic kernel $\boldsymbol{K}^{NTK}$ that governs the training dynamics of the neural network [Hron et al., 2020]:

$$\begin{aligned}\boldsymbol{K}^{NTK}(\boldsymbol{X}_S,\boldsymbol{X}'_{S'}) := \mathbb{E}_\rho\left[\phi(\boldsymbol{X}_S;\Theta)\phi(\boldsymbol{X}'_{S'};\Theta)^\top\right]\\ +\mathbb{E}_\rho\left[\nabla_\Theta\phi(\boldsymbol{X}_S;\Theta)ww^\top\nabla_\Theta\phi(\boldsymbol{X}_S;\Theta)^\top\right].\end{aligned} \tag{9}$$

The first term corresponds to the kernel induced by the attention layer's random features, analogous to the fixed kernel in Section 2.3. It measures the similarity between the context-dependent representations ϕ of the items in the choice set. The second term, on the other hand, arises from the gradients of the attention layer's parameters, capturing how pairwise interactions between items influence the utility. When the distribution of the parameters w is initialized from an independent standard Gaussian, it becomes $\mathbb{E}_\rho\left[\nabla_\Theta\phi(\boldsymbol{X}_S;\Theta)\nabla_\Theta\phi(\boldsymbol{X}_S;\Theta)^\top\right]$.

The NTK (9) mirrors the structure of the RKHS kernel $\boldsymbol{K}$ in Section 3, where predefined kernels combine set similarity and feature interactions. However, the NTK generalizes this by adaptively learning the kernel through gradient descent. The attention mechanism enables us to model context-dependent item dependencies–akin to the feature kernel in (2), but with parameters learned from data. The kernel representation of the neural network output (9) ensures that the learned utility function $\boldsymbol{u}(S,\boldsymbol{X}_S)$ resides in an RKHS implicitly defined by the network architecture, preserving the theoretical underpinnings of our framework while leveraging the flexibility of neural networks to capture nonlinear, high-dimensional patterns in choice behavior.

4 Learning the Model

In this section, we address the statistical estimation problem of learning the utility function within the RKHS. We consider three distinct learning settings: (1) learning the representation under a predefined kernel, which provides a principled starting point for modeling utility functions; (2) approximating the kernel using random feature methods, which improves computational efficiency while maintaining expressive power; and (3) learning the model with a data-adaptive kernel based on Neural Tangent Kernel (NTK) theory, enabling the kernel to adapt to the structure of the data and capture complex, high-dimensional interactions.

Our basic setup is as follows. Suppose we are given a dataset $\hat{\mathcal{D}}_N = \{(\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i}, \hat{\boldsymbol{y}}_i)\}_{i=1}^N$, where $\hat{S}_i$ is the choice set, $\hat{\boldsymbol{X}}_{\hat{S}_i}$ is the feature matrix of the items in $\hat{S}_i$, and $\hat{\boldsymbol{y}}_i$ is the observed choice vector. Denote $\boldsymbol{p}^i(\boldsymbol{u})$ to be the predicted choice probability vector for the i-th observation using the utility function $\boldsymbol{u}$. Let $\ell : \mathbb{R}^J \times \mathbb{R}^J \to \mathbb{R}_+$ be the cross-entropy loss. We consider the following empirical loss

$$\ell(\hat{\boldsymbol{y}}^i, \boldsymbol{p}^i(\boldsymbol{u})) = -\sum_{j\in\hat{S}^i}\hat{\boldsymbol{y}}^i_j u_j(\hat{S}^i,\hat{\boldsymbol{X}}_{\hat{S}_i}) + \log\sum_{k\in\hat{S}^i}\exp(u_k(\hat{S}^i,\hat{\boldsymbol{X}}_{\hat{S}_i})), \tag{10}$$

which is equivalent to log-likelihood maximization.

4.1 Learning under a Pre-defined Kernel

Consider the following choice model estimation problem

$$\min_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}}\frac{1}{N}\sum_{i=1}^{N}\ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{p}^i(\boldsymbol{u})\big),$$

In this formulation, the utility function $\boldsymbol{u}$ is not restricted to a predefined parametric form. Instead, $\boldsymbol{u}$ is learned directly in the RKHS $\mathcal{H}_{\bar{K}}$. This model-free approach eliminates restrictive assumptions about the structure of $\boldsymbol{u}$, allowing the framework to adapt to complex nonlinear relationships within the data.

Optimizing the entire RKHS with empirical log-likelihood maximization would lead to overfitting, where the model fits the training data perfectly but fails to generalize to unseen data. To address this, we introduce a RKHS norm regularization term, $\lambda \|\boldsymbol{u}\|^2_{\mathcal{H}_{\bar{K}}}$ with $\lambda > 0$, that penalizes the complexity of the utility function, ensuring that the learned model strikes a balance between fitting the training data and maintaining generalization capabilities. This leads to the following regularized risk minimization problem

$$\min_{\boldsymbol{u}\in\mathcal{H}_{\bar{K}}} \frac{1}{N}\sum_{i=1}^{N} \ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{p}^i(\boldsymbol{u})\big) + \lambda \|\boldsymbol{u}\|^2_{\mathcal{H}_{\bar{K}}}. \tag{11}$$

While the formulation in (11) provides significant flexibility, optimizing over the infinite-dimensional functional class poses a computational challenge. To address this, we leverage the Representer Theorem [Carmeli et al., 2008], which states that the optimal solution to a regularized risk minimization problem in an RKHS can be expressed as a finite linear combination of kernel functions. This result reduces the infinite-dimensional optimization problem to a finite-dimensional one, making it computationally tractable.

Theorem 1. *Problem* (11) *is equivalent to the following finite-dimensional convex optimization problem*

$$\min_{\boldsymbol{\alpha}=\{\boldsymbol{\alpha}_i\}_{i=1}^N\subset\mathbb{R}^J} \frac{1}{N}\sum_{i=1}^{N} \ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{p}^i(\boldsymbol{u}(\cdot;\boldsymbol{\alpha}))\big) + \lambda \sum_{i=1}^{N}\sum_{j=1}^{N} \boldsymbol{\alpha}_i^\top \bar{\boldsymbol{K}}\big((\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i}), (\hat{S}_j, \hat{\boldsymbol{X}}_{\hat{S}_j})\big)\boldsymbol{\alpha}_j, \tag{12}$$

where $\boldsymbol{u}(\cdot;\boldsymbol{\alpha})$ *denotes the utility function expressed in terms of the coefficients* $\boldsymbol{\alpha}$*:*

$$\boldsymbol{u}(\cdot) = \sum_{i=1}^{N} \bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big)\boldsymbol{\alpha}_i.$$

This theorem ensures that the optimal utility function $\boldsymbol{u}^\star$ lies in the span of the kernel functions evaluated at the training data points. The reformulated problem is convex because the cross-entropy loss and the regularization term are both convex in $\boldsymbol{\alpha}$. Thus, the problem becomes a finite-dimensional convex optimization over $\{\boldsymbol{\alpha}_i\}$, which can be solved using convex optimization solvers.

4.2 Learning with Random Kernel Approximation

To efficiently optimize Problem (11), we first need to determine the frequencies $\{q_m\}_m$ for approximating the kernel. A well-chosen approximation ensures that the resulting model retains the expressive power of the original kernel while significantly improving computational efficiency. To this end, we adopt the alignment and optimization formulations in [Sinha and Duchi, 2016].

Instead of using a fixed distribution of frequencies, we formulate an optimization problem that learns the best frequency distribution to align the approximated kernel with the target function. Specifically, we consider the following alignment problem:

$$\max_{\boldsymbol{q}\in\Delta_{N_w}} \sum_{i,j}\sum_{m=1}^{N_w} q_m \hat{\boldsymbol{y}}_i^\top \boldsymbol{\phi}(\hat{S}_i, X_{\hat{S}_i}; w_m)\boldsymbol{\phi}(\hat{S}_j, X_{\hat{S}_j}; w_m)^\top \hat{\boldsymbol{y}}_j, \tag{13}$$

where the goal is to optimize the frequencies over the probability simplex to maximize alignment between the approximated kernel and the true data structure. This problem can be solved efficiently and often produces a sparse solution, ensuring that the learned kernel remains computationally tractable while capturing essential dependencies in the data. A detailed proof and an alternative formulation for a one-hot choice vector are provided in Appendix B.

Once we obtain the optimal frequency distribution, we construct the corresponding feature representation. Define $\boldsymbol{\phi}(S, \boldsymbol{X}_S; \mathcal{W}_{N_w}) = [\boldsymbol{\phi}(S, \boldsymbol{X}_S; w_1)^\top; \boldsymbol{\phi}(S, \boldsymbol{X}_S; w_2)^\top; \cdots; \boldsymbol{\phi}\left(S, \boldsymbol{X}_S; w_{N_w}\right)^\top] \in \mathbb{R}^{N_w \times J}$. Given the optimized frequency weights $\hat{\boldsymbol{q}}$ to (13), we substitute this representation into the original problem (11), reducing the kernel learning task to solving the following convex optimization problem

$$\boldsymbol{\vartheta}^\star = \arg\min_{\boldsymbol{\vartheta} \in \mathbb{R}^{N_w}} \sum_{i=1}^{N} \ell\left(\hat{\boldsymbol{y}}_i, \boldsymbol{p}\big(\boldsymbol{\phi}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}; \mathcal{W}_{N_w})^\top \operatorname{diag}(\hat{\boldsymbol{q}})^{1/2} \boldsymbol{\vartheta}\big)\right) + \lambda \boldsymbol{\vartheta}^\top \boldsymbol{\vartheta}. \tag{14}$$

When $\hat{\boldsymbol{q}}$ is sparse, solving this problem only requires to store the random features corresponding to non-zero entries of $\hat{\boldsymbol{q}}$, which can be more computationally and memory-efficient.

4.3 Learning with Neural Tangent Kernel

When shifting to neural networks (8), stochastic gradient descent (SGD) is commonly employed to learn model parameters. Specifically, at iteration k, the parameters $\boldsymbol{\theta} = (\Theta, \boldsymbol{w})$ are updated as

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \hat{g}_k(z_k),$$

where $\hat{g}_k(z_k)$ is an unbiased estimator of the gradient of the loss function at $\boldsymbol{\theta}_k$ and z_k represents the randomness introduced by stochastic sampling.

In the NTK regime, neural networks behave similarly to kernel methods, enabling their training dynamics to be analyzed through a linearized, kernel-based perspective in spite of the non-convexity. This connection provides theoretical guarantees analogous to those in RKHS-based methods and classical convex optimization.

The following theorem establishes a convergence guarantee for SGD in the NTK regime, demonstrating that as the network width grows, the expected loss follows a $O(1/\sqrt{K})$ convergence rate, which aligns with standard results in convex optimization. To ease notation, we denote by $L(\boldsymbol{v}(\cdot; \boldsymbol{\theta}))$ be the value of the objective when the utility function is parameterized by $\boldsymbol{\theta}$, and by $L(\boldsymbol{v}_*)$ the optimal objective value.

Theorem 2. *Suppose the loss ℓ is Lipschitz and convex and the stochastic gradient has bounded variance. Assume $\hat{\mathcal{D}}$ consists of distinct points and $\boldsymbol{K}^{NTK}$ is universal. Let $K > 0$, $\eta = 1/\sqrt{K}$ and $\boldsymbol{\theta}_0$ be drawn from a Gaussian $\mathcal{N}(0, \sigma^2 I)$. Define $\bar{\boldsymbol{\theta}}_K$ as randomly drawn from the SGD iterates $\{\boldsymbol{\theta}_k\}_{k=0}^K$. Then as $H \to \infty$,*

$$\mathbb{E}[L(\boldsymbol{v}(\cdot; \bar{\boldsymbol{\theta}}_K))] \leq L(\boldsymbol{v}_*) + \frac{C}{\sqrt{K}} + o_p(1),$$

where the expectation is taken over the randomness of stochastic gradients and the uniform sampling from iterates, and the high probability is with respect to the initialization.

This result establishes that, as the number of iterations K increases, the expected loss approaches the optimal loss at a rate of $\sqrt{K}$, reflecting the standard convergence behavior of SGD in convex settings. The additional $o_p(1)$ term, which vanishes in probability, captures effects that become negligible as the width of the network grows.

Thereby, the NTK framework provides a rigorous link between neural network training dynamics and kernel methods, reinforcing the idea that overparameterized networks can be effectively analyzed through a linearized, kernel-based perspective.

4.4 Generalization Error Bounds

A fundamental question in learning theory is how well a trained model generalizes to unseen data. The generalization error quantifies the difference between the expected loss on the true data distribution and the empirical loss observed on the training set. Understanding generalization is crucial for ensuring that models not only fit the training data but also perform well on new inputs.

In kernel-based methods, generalization is often controlled by the RKHS norm of the learned function. Intuitively, a smaller RKHS norm corresponds to a smoother function, which tends to generalize better. We are interested in how the constraint on the RKHS norm contributes to the model's generalization capability. The regularized problem in (11) facilitates the control of $\|\boldsymbol{u}\|_{\mathcal{H}_{\boldsymbol{K}}}$.

Let $\mathcal{D}$ be the underlying true distribution of $(S, \boldsymbol{X}_S, \boldsymbol{y})$. The theorem below provides an upper bound of the generalization error (a.k.a the difference between the expected risk and empirical risk) for the RKHS model with norm control.

Theorem 3. *Suppose* $\mathbb{E}_{\mathcal{D}}\left[\bar{\boldsymbol{K}}_{jj}\big((S, \boldsymbol{X}_S), (S, \boldsymbol{X}_S)\big)\right] \le \sigma^2$ *for any* $j \in [J]$. *With probability at least* $1-\delta$, *for all* $\boldsymbol{u} \in \mathcal{H}_{\bar{\boldsymbol{K}}}$ *satisfying* $\|\boldsymbol{u}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \le \tau$ *and* $\|\boldsymbol{u}\|_\infty \le R$,

$$\mathbb{E}_{\mathcal{D}}\left[\ell\big(\boldsymbol{y}, \boldsymbol{u}(S, \boldsymbol{X}_S)\big)\right] \le \frac{1}{N}\sum_{i=1}^{N} \ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{u}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big) + 4\tau\sigma\sqrt{\frac{J}{N}} + (\log(J) + 2R)\sqrt{\frac{\ln(1/\delta)}{2N}}.$$

Theorem 3 establishes a generalization error of order $O(\sqrt{J/N})$. It does not suffer from the curse of dimensionality in feature dimensions. When the maximum cardinality of the choice set, J, is significantly smaller than the training data size, N, the error order becomes inversely proportional to the square root of N. This suggests that our model's generalization capability remains robust to increasing kernel complexity, provided that the RKHS norm is properly controlled. Consequently, this justifies our exploration of more complex nonlinear kernels in the previous section.

We remark that this result holds not only for the fixed kernels described in Sections 3.1 and 3.2, but also for random kernel in Section 3.3 and NTK in Section 3.4, so long as the RKHS norm of these kernels are bounded (with high probabiility).

5 Numerical Experiements

We evaluate the performance of the RKHS choice models using both synthetic datasets and two real-world datasets tailored to featureless and feature-based approaches respectively.

Given the dataset $\{(\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i}, \hat{\boldsymbol{y}}_i)\}_{i=1}^N$, we use two types of performance metrics.

(1) Negative log-likelihood (NLL), equivalent to the cross-entropy loss (10):

$$\text{NLL} = -\sum_{i=1}^{N} \hat{\boldsymbol{y}}_i \log p^i(\boldsymbol{u});$$

(2) Root mean square error (RMSE) along with Soft-RMSE if ground truth is available [Berbeglia et al., 2022]:

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^N \|\hat{\boldsymbol{y}}_i - p^i(\boldsymbol{u})\|_2^2}{\sum_{i=1}^N (|\hat{S}_i| + 1)}},$$

$$\text{Soft-RMSE} = \sqrt{\frac{\sum_{i=1}^N \|\boldsymbol{y}_i^* - p^i(\boldsymbol{u})\|_2^2}{\sum_{i=1}^N (|\hat{S}_i| + 1)}},$$

where $p^i(\boldsymbol{u})$ is the predicted choice probability for i-th data sample, and $\boldsymbol{y}_i^*$ is the true choice vector computed by the known ground truth. RMSE is used in the real data while Soft-RMSE is used in the synthetic data for out-of-sample performance evaluation.

All hyperparameters of the stationary kernel functions are initialized based on standard settings. Further fine-tuning could be performed based on validation data to optimize model performance. See Appendix F.2 for more details.

5.1 Synthetic Experiment: Context Effects

A series of synthetic experiments are designed to show the capability of our model in capturing complex context effects, specifically, the p-order interaction effects. This serves as a numerical companion of our theoretical result (Proposition 2). The ground truth choice model is set as a p-order Halo effect model defined in (4), where the base utility and interaction coefficients are pre-specified as detailed in the Appendix D. The experiment is under two folds: i) When sufficient data is available, we assess the expressiveness of our model compared to other benchmarks. ii) With limited training data, we evaluate the generalization capability of our kernel-based model against regression, assuming the true model and correct order are known.

5.1.1 First- and Second-order Interactions

Consider $J = 5$ items in the universe. We generated $N = 6,400$ observations by first uniformly sampling each choice set and then obtaining the respective choice using the pre-determined ground truth model.

We employed the polynomial kernel with $p = 1$ and $p = 2$, which by Proposition 2 should capture interactions up to the first order and second order respectively. We compare our model (RKHS), with two other benchmarks that also consider the context effect, Unmasked-RKHS [Ángel Martín-Baos et al., 2024] and Context MNL (CMNL) [Yousefi Maragheh et al., 2020]. Unmasked-RKHS is also set to use the p-order polynomial kernel but without the masking mechanism, whereas CMNL mainly models the first-order interactions.

Table 1. Comparison Methods for Modeling Interactions

	First-order		Second-order	
Model	In-sample NLL	Soft-RMSE	In-sample NLL	Soft-RMSE
CMNL	0.75211	0.01025	0.75437	0.01770
RKHS (Unmasked)	0.75230	0.01020	0.75232	0.01034
RKHS	**0.75185**	**0.01018**	**0.75192**	**0.01030**

Table 1 summarizes the in-sample performance (NLL) and out-of-sample performance (Soft-RMSE). We have the following observations:

(1) Overall, our model shows a superior performance in capturing the in-sample Halo effect for both interaction orders.

(2) Our introduced masking mechanism provides extra benefits.

(3) With second-order interaction involved, CMNL exhibits a significant performance gap compared to the other two models, as it does not account for higher-order interactions.

5.1.2 High-order Halo effects with Limited Data

To demonstrate the efficiency of our RKHS approach in modeling the Halo effect with limited data, we conduct the following experiment.

Consider a scenario with N transactions across J distinct products, where the ground truth follows a logit model with a third-order interaction effect. The dataset generation process also follows a uniformly random fashion as in the previous experiment except that one choice set is generated for every 10 transactions. This setup reflects the practical situation where a limited number of assortments were tested across a time period and the same assortment was shown to multiple consumers.

Assume that we know the ground truth is a p-order polynomial interaction model and $p = 3$. Using (4), a simple regression with MNL output can be employed to estimate each coefficient in the interaction model. We term this method as PolyFit. When the data is limited such that $N \ll O(J^3)$, PolyFit requires estimating a total of $J + \binom{J}{2} + \sum_{i=2}^{3} J \cdot \binom{J-1}{i} = O(J^4)$ parameters. In contrast, our kernel model has $J \cdot N$ parameters, resulting in significantly lower model complexity. We consider $N \in \{250, 500, 1{,}000\}$, and $J = 20$ and summarize the result in Table 2.

Table 2. Comparison of PolyFit and RKHS on Limited Data. The percentages record the percentage changes of the performance measure of RKHS compared to that of PolyFit.

		# of Parameters		In-sample NLL		Soft-RMSE	
N	J	PolyFit	RKHS	PolyFit	RKHS	PolyFit	RKHS
250	20	23,010	5,000	0.78606	0.81178 (+3.27%)	0.10055	0.09583 (-4.69%)
500	20	23,010	10,000	0.80090	0.81659 (+1.96%)	0.07510	0.07182 (-4.37%)
1000	20	23,010	20,000	0.87021	0.89088 (+2.38%)	0.07450	0.06963 (-6.54%)

The results reveal the following insights:

(1) The PolyFit model generally achieves lower training loss (NLL) that RKHS. This suggests that PolyFit optimizes well on the training set, likely due to the larger model complexity and no model misspecification bias, whereas the regularization in RKHS introduces a small amount of bias.

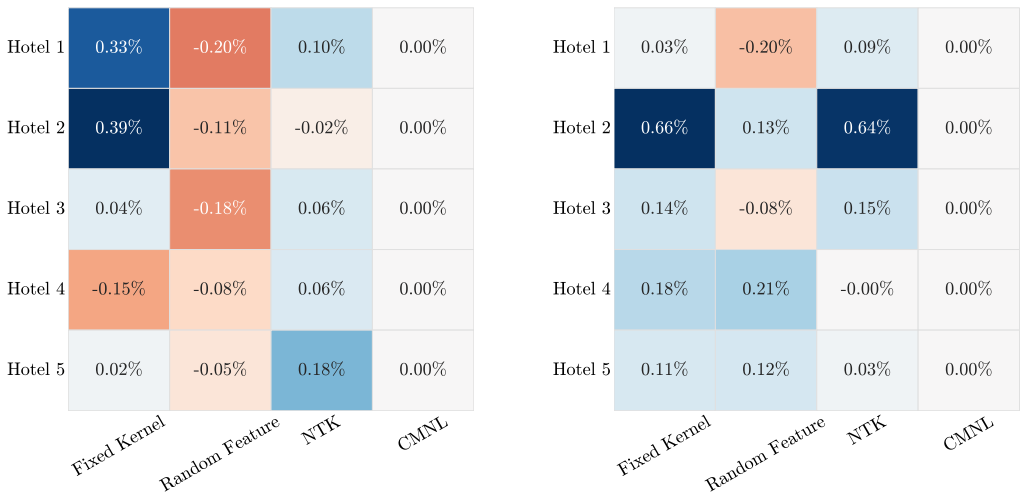
(2) In contrast, the test performance (Soft-RMSE) depicts a different pattern. RKHS consistently achieves lower Soft-RMSE than PolyFit, indicating better superior generalization ability. This indicates our nonparametric kernel model can more efficiently capture the true interaction pattern with fewer parameters and limited information. Moreover, compared to PolyFit, our model's relatively large decrease in out-of-sample error compensates for its relatively small in-sample performance gap.

5.2 Datasets without Features

The Hotel Dataset [Bodea et al., 2009] consists of booking transactions from five hotels from a major hotel chain. We follow the preprocessing step widely adopted in the literature [Berbeglia et al., 2022, Şimşek and Topaloglu, 2018, Van Ryzin and Vulcano, 2015], obtaining the featureless dataset for each hotel; see Appendix E.1 for more descriptions. An *item* in this dataset is defined as a room type, such as King room or Double room, or the no-purchase option. Each choice set contains the available room types plus the no-purchase option at a transaction. Table 3 provides a description of the basic information of the preprocessed dataset.

Table 3. Preprocessed Hotel Data

	J (including no-purchase option)	In-sample transactions	Out-of-sample transactions
Hotel 1	11	5,290	1,325
Hotel 2	11	1,845	465
Hotel 3	9	5,070	1,270
Hotel 4	5	1,100	275
Hotel 5	7	1,000	255



(a) In-Sample Performance (b) Out-of-Sample Performance

Fig. 1. Benchmark gap difference of RMSE in ratio with respect to the best kernel across 5 hotels (The blue color indicates better performance) .

Figure 1 compares the RMSE gaps between our RKHS models (including Fixed Kernel, Random Feature with $N_w = 2 \times 10^4$, NTK with $H = 1,024$) and the featureless benchmark CMNL [Yousefi Maragheh et al., 2020] which consider first-order Halo effect. For the Fixed Kernel, we reported the result of the best-performed one out of linear, Sigmoid, polynomial, Gaussian, and Matérn kernels; see a full result in the Appendix Table 11. In order to observe possible improvement when considering the more complex context effect beyond first-order Halo effect, this heat map visualization encodes the relative deviations from the benchmark CMNL, where cooler (in blue) and warmer (in red) colors represent larger improvements and gaps with this benchmark respectively.

Our findings are concluded as follows.

(1) Compared to CMNL, our RKHS-based methods, except Random Feature, generally enhance the in-sample performance, which reflects the importance of modeling more complicated (e.g., higher-order) item interactions. The performance limitation of Random Feature may be due to the approximation error. Furthermore, NTK consistently improves or maintains

the relative in-sample performance across all five hotels, highlighting the adaptability of neural kernels to diverse datasets.

(2) The out-of-sample performance of our methods is generally better than CMNL, indicating that our RKHS methods possess better generalization capability, whereas the CMNL method might exhibit overfitting. Moreover, Random Feature approximates better in smaller datasets, as observed in Hotel 4 and Hotel 5.

(3) Memory efficiency: Despite the superior in-sample performance of Fixed Kernel in large datasets (Hotel 1 and Hotel 2), the memory bottleneck exists even with mixed-precision training. Both Random Feature and NTK are introduced to address this challenge. Out of these two resolutions, NTK is considered better because the advantages of in-sample and out-of-sample performance are retained.

5.3 Datasets with Features

- Swissmetro: The Swissmetro dataset [Bierlaire et al., 2001], used for the feature-based model, is derived from a survey conducted in Switzerland to assess the potential demand for a new mode of public transportation called Swissmetro. An *item* in this dataset is defined as a mode of transportation. The available transportation options can include Swissmetro, train and car. For each observation, the feature vector is constructed by integrating continuous and discrete attributes of the transportation mode, such as offered transportation, travel time, ticket type, and origin and destination, with customer-specific characteristics.
- Expedia: The Expedia datast, publicly available at Kaggle, presents a more complex and heterogeneous setting. Here, an item is defined as a hotel property. Each choice set corresponds to the hotels presented to a user during a search session, where both the size and composition of the set vary across observations. The feature representation incorporates both hotel-specific variables (e.g., location score, star rating, historical price) and user-specific information (e.g., search destination, booking window, user historical behavior). The resulting dataset reflects a realistic and large-scale online recommendation scenario, with sparsity and personalization challenges.

Table 4 provides an overview of the two datasets used in our feature-based models. Details of the preprocessing steps, adapted from Aouad and Désir [2023], are given in Appendix E.2.

Property	Swissmetro	Expedia
Largest Assortment Size	3	39
Mean Assortment Size	2.8	25.8
No-purchase Option	No	Yes
Sample Size	10719	387618
Feature Dimension	89	91

Table 4. Summary of Feature Datasets

Table 5. Predictive performance on Swissmetro dataset (in-sample vs. out-of-sample)

	Model	In-sample		Out-of-sample	
		NLL	Accuracy	NLL	Accuracy
Context-Independent	MNL	0.837	0.624	0.837	0.623
	TasteNet	0.385	0.846	0.558	0.790
	Vanilla Neural Network	0.434	0.823	0.581	0.759
	RUMnet	0.324	0.874	0.546	0.797
Context-Dependent	RKHS(0.5-Matérn)	**0.293**	**0.900**	**0.381**	**0.845**
	Random Feature ($N_w = 2 \times 10^4$)	0.417	0.840	0.461	0.811
	NTK ($H = 32$)	0.418	0.832	0.483	0.800
	NTK ($H = 1,000$)	0.381	0.847	0.436	0.816
	DLCL	0.681	0.702	0.679	0.680
	FATENet	0.413	0.833	0.483	0.794
	TCNet	0.434	0.816	0.490	0.789

Table 6. Predictive performance on Expedia dataset (in-sample vs. out-of-sample)

	Model	In-sample		Out-of-sample	
		NLL	Accuracy	NLL	Accuracy
Context-Independent	MNL	2.563	0.307	2.563	0.307
	TasteNet	2.111	0.407	2.114	0.407
	Vanilla Neural Network	2.644	0.318	2.679	0.313
	RUMnet	2.002	0.428	2.019	0.425
Context-Dependent	NTK ($H = 32$)	2.095	0.407	2.096	0.408
	NTK ($H = 1000$)	2.058	0.419	2.060	0.419
	Deep NTK($H = 16$, $L = 10$)	**1.511**	**0.554**	**1.764**	**0.519**
	DLCL	2.062	0.416	2.063	0.416
	FATENet	2.001	0.430	2.007	0.429
	TCNet	2.128	0.401	2.128	0.401

Table 5 and 6 report the predictive performance of our RKHS-based models in comparison with established benchmarks, including context-independent benchmarks taken from Aouad and Désir [2023] and serveral context-dependent benchmarks: DLCL([Tomlinson and Benson, 2020]), FATENet (Lea [2022]), TCNet(Wang et al. [2023b]). We conclude our key findings as follows:

(1) **Our RKHS-based methods consistently outperform all benchmark models**. Across both datasets, they achieve the best in-sample NLL and accuracy. On the smaller Swissmetro dataset, the Matérn kernel achieves an out-of-sample NLL of 0.381 and accuracy of 0.845, representing a 21.12% reduction in NLL and a 6.02% improvement in accuracy compared to the best benchmark model. For the larger dataset Expedia, where the fixed kernel method becomes computationally infeasible, DeepNTK achieves and out-of-sample NLL of 1.764 and accuracy of 0.519, representing a 12.10% reduction in NLL and 20.98% improvement in accuracy over the strongest baseline. These results highlight the scalability and effectiveness of RKHS-based methods in capturing complex context-dependent choice behavior across varying dataset sizes.

(2) **Kernels with lower smoothness and attention-based NTK structures effectively capture preference irregularities**. The Matérn kernel with small smoothness parameter ν as well as the NTK with an attention structure can effectively capture the inherent roughness which proves advantageous in Swissmetro dataset as shown in Appendix Table 14, increasing ν degrades both in-sample and out-of-sample NLL performance. One explanation could be the abrupt preference changes between transportation modes require localized sensitivity rather than global smoothness.

(3) **More heads enhances model expressiveness**. Increasing the number of heads, H, in the attention layer to a large value improves the performance significantly (with 17.3% in-sample and 10.1% out-of-sample loss decrease on Swissmetro). This result is consistent with Theorem 2 where the objective converges globally as H approaches infinity. However, the effect is less pronounced on the Expedia dataset, possibly due to its scale and complexity diluting the marginal gains from increased attention diversity.

(4) **More layers substantially improve model capacity.** More layers substantially improve model capacity. Increasing the number of layers in the NTK model results in significant gains in both in-sample and out-of-sample performance, while also helping to reduce the total number of parameters. For example, on the Expedia dataset, the deep NTK model with 10 layers and only 16 heads achieves a 14.37% reduction in out-of-sample NLL and a 23.87% improvement in accuracy compared to the wide NTK model with 1000 heads. This demonstrates that increasing depth (more layers) is more effective than simply increasing width (more heads) in enhancing model expressiveness.

6 Conclusion

This paper presents a comprehensive study of RKHS-based discrete choice models from three key perspectives: modeling, computation, and statistical generalization. On the modeling front, we introduce a vector-valued RKHS framework that captures context-dependent choice behavior beyond the constraints of traditional random utility models. From a computational perspective, we develop efficient learning algorithms with provable convergence guarantees. Statistically, we establish generalization error bounds, ensuring that our model balances complexity and predictive performance. By unifying insights from kernel methods, machine learning, and discrete choice theory, our work provides a principled, scalable framework for capturing rich choice behaviors.

Future directions include exploring dynamic choice modeling and further integrating behavioral insights into RKHS-based frameworks.

References

2022. Learning Context-Dependent Choice Functions. *International Journal of Approximate Reasoning* 140 (Jan. 2022), 116–155. https://doi.org/10.1016/j.ijar.2021.10.002

Yi-Chun Akchen and Dmitry Mitrofanov. 2024. Consider or Choose? The Role and Power of Consideration Sets. http://arxiv.org/abs/2302.04354 arXiv:2302.04354.

Aydın Alptekinoğlu and John H Semple. 2016. The exponomial choice model: A new alternative for assortment and price optimization. *Operations Research* 64, 1 (2016), 79–93.

José Ángel Martín-Baos, Ricardo García-Ródenas, Luis Rodriguez-Benitez, and Michel Bierlaire. 2024. Scalable Kernel Logistic Regression with Nystr\" om Approximation: Theoretical Analysis and Application to Discrete Choice Modelling. *arXiv e-prints* (2024), arXiv–2402.

Ali Aouad and Antoine Désir. 2023. Representing Random Utility Choice Models with Neural Networks. https://doi.org/10.48550/arXiv.2207.12877 arXiv:2207.12877 [cs, math, stat].

Ali Aouad, Vivek Farias, and Retsef Levi. 2021. Assortment Optimization Under Consider-Then-Choose Choice Models. *Management Science* 67, 6 (June 2021), 3368–3386. Publisher: INFORMS.

Richard R. Batsell and John C. Polking. 1985. A New Class of Market Share Models. *Marketing Science* 4, 3 (1985), 177–198. https://www.jstor.org/stable/183903 Publisher: INFORMS.

Gerardo Berbeglia, Agustín Garassino, and Gustavo Vulcano. 2022. A Comparative Empirical Study of Discrete Choice Models in Retail Operations. *Management Science* 68, 6 (June 2022), 4005–4023. https://doi.org/10.1287/mnsc.2021.4069

Michel Bierlaire, Kay Axhausen, and Georg Abay. 2001. The acceptance of modal innovation: The case of Swissmetro. In *Swiss transport research conference*.

Jose Blanchet, Guillermo Gallego, and Vineet Goyal. 2016. A Markov chain approximation to choice modeling. *Operations Research* 64, 4 (2016), 886–905.

Tudor Bodea, Mark Ferguson, and Laurie Garrow. 2009. Data Set—Choice-Based Revenue Management: Data from a Major Hotel Chain. *Manufacturing & Service Operations Management* 11, 2 (2009), 356–361.

Richard L. Brady and John Rehbeck. 2016. Menu-Dependent Stochastic Feasibility. *Econometrica* 84, 3 (2016), 1203–1223. https://www.jstor.org/stable/43866462 Publisher: [Wiley, The Econometric Society].

D Brownstone. 2000. Discrete choice modeling for transportation. In *International conference on travel behaviour research, 9th, 2000, Gold Coast, Queensland, Australia, vol 1*.

C. Carmeli, E. De Vito, A. Toigo, and V. Umanità. 2008. Vector valued reproducing kernel Hilbert spaces and universality. http://arxiv.org/abs/0807.1659 arXiv:0807.1659 [math].

Yoosoon Chang, Steven N Durlauf, Bo Hu, and Joon Park. 2025. *Accounting for Individual-Specific Heterogeneity in Intergenerational Income Mobility*. Technical Report. National Bureau of Economic Research.

Pradeep K Chintagunta and Harikesh S Nair. 2011. Structural workshop paper—discrete-choice models of consumer demand in marketing. *Marketing Science* 30, 6 (2011), 977–996.

José Luis Espinosa-Aranda, Ricardo García-Ródenas, Maria del Carmen Ramirez-Flores, María Luz López-García, and Eusebio Angulo. 2015. High-speed railway scheduling based on user preferences. *European Journal of Operational Research* 246, 3 (2015), 772–786.

José Luis Espinosa-Aranda, Ricardo García-Ródenas, María Luz López-García, and Eusebio Angulo. 2018. Constrained nested logit model: formulation and estimation. *Transportation* 45 (2018), 1523–1557.

Vivek F Farias, Srikanth Jagabathula, and Devavrat Shah. 2013. A nonparametric approach to modeling choice with limited data. *Management science* 59, 2 (2013), 305–322.

Guillermo Gallego and Mina M. Iravani. 2024. Axioms of Rationality for Discrete Choice Models. https://doi.org/10.2139/ssrn.4790254

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

Jiri Hron, Yasaman Bahri, Jascha Sohl-Dickstein, and Roman Novak. 2020. Infinite attention: NNGP and NTK for deep attention networks. In *International Conference on Machine Learning*. PMLR, 4376–4386.

Joel Huber, John W. Payne, and Christopher Puto. 1982. Adding Asymmetrically Dominated Alternatives: Violations of Regularity and the Similarity Hypothesis. *Journal of Consumer Research* 9, 1 (June 1982), 90–98. https://doi.org/10.1086/208899 Publisher: Oxford Academic.

Arthur Jacot, Franck Gabriel, and Clément Hongler. 2018. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems* 31 (2018).

Srikanth Jagabathula and Paat Rusmevichientong. 2018. The Limit of Rationality in Choice Modeling: Formulation, Computation, and Implications. *Management Science* (Aug. 2018), mnsc.2018.3030. https://doi.org/10.1287/mnsc.2018.3030

Daniel Kahneman and Amos Tversky. 1979. Prospect Theory: An Analysis of Decision under Risk. *Econometrica* 47, 2 (1979), 263–291. https://doi.org/10.2307/1914185 Publisher: [Wiley, Econometric Society].

Joohwan Ko and Andrew A. Li. 2024. Modeling Choice via Self-Attention. http://arxiv.org/abs/2311.07607 arXiv:2311.07607 [cs].

Zihao Li, Zhuoran Yang, and Mengdi Wang. 2023. Reinforcement learning with human feedback: Learning dynamic choices via pessimism. *arXiv preprint arXiv:2305.18438* (2023).

R Duncan Luce. 1959. *Individual choice behavior*. Vol. 4. Wiley New York.

Charles F. Manski. 1977. The structure of random utility models. *Theory and Decision* 8, 3 (July 1977), 229–254. https://doi.org/10.1007/BF00133443

Reza Yousefi Maragheh, Alexandra Chronopoulou, and James Mario Davis. 2018. A Customer Choice Model with HALO Effect. http://arxiv.org/abs/1805.01603 arXiv:1805.01603 [stat].

Jacob Marschak. 1959. *Binary Choice Constraints on Random Utility Indicators*. Technical Report. Cowles Foundation for Research in Economics, Yale University.

José Ángel Martín-Baos, Ricardo García-Ródenas, and Luis Rodriguez-Benitez. 2021a. A Python package for performing penalized maximum likelihood estimation of conditional logit models using Kernel Logistic Regression. *Transportation research procedia* 58 (2021), 61–68.

José Ángel Martín-Baos, Ricardo García-Ródenas, and Luis Rodriguez-Benitez. 2021b. Revisiting kernel logistic regression under the random utility models perspective. An interpretable machine-learning approach. *Transportation Letters* 13, 3 (2021), 151–162.

José Ángel Martín-Baos, Ricardo García-Ródenas, Luis Rodriguez-Benitez, and Michel Bierlaire. 2025. Scalable Kernel Logistic Regression with Nyström Approximation: Theoretical Analysis and Application to Discrete Choice Modelling. *Neurocomputing* 617 (Feb. 2025), 128975. https://doi.org/10.1016/j.neucom.2024.128975 arXiv:2402.06763 [cs].

Andreas Maurer. 2016. A vector-contraction inequality for rademacher complexities. In *Algorithmic Learning Theory: 27th International Conference, ALT 2016, Bari, Italy, October 19-21, 2016, Proceedings 27*. Springer, 3–17.

D McFadden. 1974a. Conditional Logit Analysis of Qualitative Choice Behavior. *Frontiers in Econometrics* (1974).

Daniel McFadden. 1974b. The measurement of urban travel demand. *Journal of public economics* 3, 4 (1974), 303–328.

Daniel McFadden. 1986. The choice theory approach to market research. *Marketing science* 5, 4 (1986), 275–297.

Daniel McFadden. 2001. Economic Choices. *The American Economic Review* 91, 3 (2001), 351–378. https://www.jstor.org/stable/2677869 Publisher: American Economic Association.

Daniel McFadden, Antti Talvitie, Stephen Cosslett, Ibrahim Hasan, Michael Johnson, Fred Reid, and Kenneth Train. 1977. Demand model estimation and validation. *Urban Travel Demand Forecasting Project, Phase* 1 (1977).

Daniel McFadden and Kenneth Train. 2000. Mixed MNL models for discrete response. *Journal of applied Econometrics* 15, 5 (2000), 447–470.

Richard E Nisbett and Timothy D Wilson. 1977. The halo effect: Evidence for unconscious alteration of judgments. *Journal of personality and social psychology* 35, 4 (1977), 250.

Karlson Pfannschmidt, Pritha Gupta, Björn Haddenhorst, and Eyke Hüllermeier. 2022. Learning context-dependent choice functions. *International Journal of Approximate Reasoning* 140 (2022), 116–155.

Ali Rahimi and Benjamin Recht. 2007. Random features for large-scale kernel machines. *Advances in neural information processing systems* 20 (2007).

Nir Rosenfeld, Kojin Oshiba, and Yaron Singer. 2020. Predicting choice with set-dependent aggregation. In *International Conference on Machine Learning*. PMLR, 8220–8229.

Arjun Seshadri, Alex Peysakhovich, and Johan Ugander. 2019. Discovering context effects from raw choice data. In *International conference on machine learning*. PMLR, 5660–5669.

Shai Shalev-Shwartz and Shai Ben-David. 2014. *Understanding machine learning: From theory to algorithms*. Cambridge university press.

Itamar Simonson. 1989. Choice Based on Reasons: The Case of Attraction and Compromise Effects. *Journal of Consumer Research* 16, 2 (Sept. 1989), 158–174. https://doi.org/10.1086/209205

A Serdar Şimşek and Huseyin Topaloglu. 2018. An expectation-maximization algorithm to estimate the parameters of the Markov chain choice model. *Operations Research* 66, 3 (2018), 748–760.

Rahul Singh and Suhas Vijaykumar. 2023. Kernel ridge regression inference with applications to preference data. *arXiv preprint arXiv:2302.06578* (2023).

Aman Sinha and John C Duchi. 2016. Learning kernels with random features. *Advances in neural information processing systems* 29 (2016).

Kalyan Talluri and Garrett Van Ryzin. 2004. Revenue management under a general discrete choice model of consumer behavior. *Management Science* 50, 1 (2004), 15–33.

Louis L Thurstone. 1927. A law of comparative judgment. *Psychological review* (1927).
Kiran Tomlinson and Austin R. Benson. 2020. Learning Interpretable Feature Context Effects in Discrete Choice. *CoRR* abs/2009.03417 (2020). arXiv:2009.03417 https://arxiv.org/abs/2009.03417
Amos Tversky. 1972. Elimination by aspects: A theory of choice. *Psychological Review* 79, 4 (1972), 281–299. https://doi.org/10.1037/h0032955 Place: US Publisher: American Psychological Association.
Amos Tversky and Daniel Kahneman. 1991. Loss Aversion in Riskless Choice: A Reference-Dependent Model*. *The Quarterly Journal of Economics* 106, 4 (Nov. 1991), 1039–1061. https://doi.org/10.2307/2937956
Amos Tversky and Itamar Simonson. 1993. Context-Dependent Preferences. *Management Science* 39, 10 (Oct. 1993), 1179–1189. https://doi.org/10.1287/mnsc.39.10.1179 Publisher: INFORMS.
Garrett Van Ryzin and Gustavo Vulcano. 2015. A market discovery algorithm to estimate a general class of nonparametric choice models. *Management Science* 61, 2 (2015), 281–300.
A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
Hanzhao Wang, Zhongze Cai, Xiaocheng Li, and Kalyan Talluri. 2023a. A neural network based choice model for assortment optimization. *arXiv preprint arXiv:2308.05617* (2023).
Hanzhao Wang, Xiaocheng Li, and Kalyan Talluri. 2023b. Transformer Choice Net: A Transformer Neural Network for Choice Prediction. http://arxiv.org/abs/2310.08716 arXiv:2310.08716 [cs].
Ruxian Wang. 2021. Discrete choice models and applications in operations management. In *Tutorials in Operations Research: Emerging Optimization Methods and Modeling Techniques with Applications*. INFORMS, 199–226.
Ruxian Wang. 2022. The Threshold Effects on Consumer Choice and Pricing Decisions. *Manufacturing & Service Operations Management* 24, 1 (Jan. 2022), 448–466. https://doi.org/10.1287/msom.2020.0948 Publisher: INFORMS.
Huw CWL Williams. 1977. On the formation of travel demand models and economic evaluation measures of user benefit. *Environment and planning A* 9, 3 (1977), 285–344.
Reza Yousefi Maragheh, Xin Chen, James Davis, Jason Cho, and Sushant Kumar. 2020. Choice Modeling and Assortment Optimization in the Presence of Context Effects. *SSRN Electronic Journal* (2020). https://doi.org/10.2139/ssrn.3747354
Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. 2017. Deep sets. *Advances in neural information processing systems* 30 (2017).
Tong Zhang. 2023. *Mathematical analysis of machine learning algorithms*. Cambridge University Press.

A Proofs

This appendix contains the proofs of the propositions and theorems presented in the main text.

A.1 Proof of Proposition 1

Proof. Let $\{\boldsymbol{e}_j\}_{j=1}^J$ denote the standard basis in $\mathbb{R}^J$. By the reproducing property,

$$\langle \boldsymbol{u}, \boldsymbol{K}^{\mathcal{S}}(\cdot, S)\boldsymbol{\alpha}\rangle_{\mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}} = \boldsymbol{u}(S)^\top \boldsymbol{\alpha}.$$

Then the component $u_j(S)$ can be expressed as

$$u_j(S) = \boldsymbol{u}(S)^\top \boldsymbol{e}_j = \langle \boldsymbol{u}, \boldsymbol{K}^{\mathcal{S}}(\cdot, S)\boldsymbol{e}_j\rangle_{\mathcal{H}_{\boldsymbol{K}}}, \quad \forall j \in \{1, \ldots, J\}. \tag{15}$$

Additionally, $\boldsymbol{\alpha}_i$ can be written as

$$\boldsymbol{\alpha}_i = \sum_{j=1}^J \alpha_{i,j}\boldsymbol{e}_j, \quad \forall i \in \{1, \ldots, m\}. \tag{16}$$

Recall the RKHS norm is defined via the RKHS inner product (1) where $\|\boldsymbol{u}\|_{\mathcal{H}}^2 = \langle \boldsymbol{u}, \boldsymbol{u}\rangle_{\mathcal{H}}$. Substituting (16) in this definition, we obtain

$$\begin{aligned}\|\boldsymbol{u}\|_{\mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}}^2 &= \left\langle \boldsymbol{u}, \sum_{i=1}^m \boldsymbol{K}^{\mathcal{S}}(\cdot, S_i)\boldsymbol{\alpha}_i \right\rangle_{\mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}} \\ &= \left\langle \boldsymbol{u}, \sum_{i=1}^m \sum_{j=1}^J \alpha_{i,j}\boldsymbol{K}^{\mathcal{S}}(\cdot, S_i)\boldsymbol{e}_j \right\rangle_{\mathcal{H}_{\boldsymbol{K}^{\mathcal{S}}}}.\end{aligned}$$

Expanding the sum, we get

$$\|\boldsymbol{u}\|^2_{\mathcal{H}_{\boldsymbol{K}^S}} = \sum_{i=1}^{m}\sum_{j=1}^{J}\left\langle \boldsymbol{u}, \boldsymbol{K}^S(\cdot, S_i)\alpha_{i,j}\boldsymbol{e}_j\right\rangle_{\mathcal{H}_{\boldsymbol{K}^S}}.$$

By substituting (15), we obtain

$$\|\boldsymbol{u}\|_{\mathcal{H}_{\boldsymbol{K}^S}} = \sum_{i=1}^{m}\sum_{j=1}^{J} u_j(S_i)\alpha_{i,j}.$$

Since $u_j(S) = 0$ for $j \notin S$ by the masked kernel in (2), this simplifies to

$$\|\boldsymbol{u}\|^2_{\mathcal{H}_{\boldsymbol{K}^S}} = \sum_{i=1}^{m}\sum_{j\in S_i} u_j(S_i)\alpha_{i,j},$$

which indicates $\|\boldsymbol{u}\|_{\mathcal{H}_{\boldsymbol{K}^S}}$ is independent of irrelevant $u_j(S)$ for $j \notin S$. □

A.2 Proof of Theorem 1

Proof. Let $\boldsymbol{u} \in \mathcal{H}_{\bar{\boldsymbol{K}}}$, and decompose it into

$$\boldsymbol{u} = \boldsymbol{u}_P + \boldsymbol{u}_R,$$

where $\boldsymbol{u}_P$ is in the finite-dimensional subspace spanned by $\left\{\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big) : i = 1, \ldots, N\right\}$ and $\boldsymbol{u}_R$ is in the orthogonal complement of this subspace. Since $\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big)$ reproduces function evaluations, we have

$$\boldsymbol{u}(\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i}) = \left\langle \boldsymbol{u}_R + \boldsymbol{u}_P, \bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big)\right\rangle = \left\langle \boldsymbol{u}_R, \bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big)\right\rangle = \boldsymbol{u}_R(\hat{\boldsymbol{X}}_{\hat{S}_i}).$$

As a result, for the probability function $\boldsymbol{p}$, we also have

$$\boldsymbol{p}^i(\boldsymbol{u}) = \boldsymbol{p}^i(\boldsymbol{u}_P).$$

This shows that the loss term $\frac{1}{N}\sum_{i=1}^{N}\ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{p}^i(\boldsymbol{u})\big)$ depends only on $\boldsymbol{u}_P$. Moreover, by the Pythagoream theorem

$$\|\boldsymbol{u}\|^2_{\mathcal{H}_{\bar{\boldsymbol{K}}}} = \|\boldsymbol{u}_P\|^2_{\mathcal{H}_{\bar{\boldsymbol{K}}}} + \|\boldsymbol{u}_R\|^2_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \geq \|\boldsymbol{u}_P\|^2_{\mathcal{H}_{\bar{\boldsymbol{K}}}}.$$

Therefore, minimizing the regularized risk requires $\boldsymbol{u}_R = 0$, implying that the minimizer $\boldsymbol{u}^\star$ lies entirely in the finite-dimensional subspace. Hence, it can be written as

$$\boldsymbol{u}^\star(\cdot) = \sum_{i=1}^{N}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \hat{\boldsymbol{X}}_{\hat{S}_i})\big)\boldsymbol{\alpha}_i^\star$$

where $\boldsymbol{\alpha}_i^\star \in \mathbb{R}^J$, $i = 1, \ldots, N$, N is finite. □

A.3 Proof of Theorem 3

We define the vector-valued function class

$$\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau) = \{\boldsymbol{u} \in \mathcal{H}_{\bar{\boldsymbol{K}}}, \|\boldsymbol{u}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \leq \tau\}. \tag{17}$$

The function class of the scalar-valued loss function associated with $\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)$ is denoted by

$$\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau) = \left\{\ell\big(\boldsymbol{y}, \boldsymbol{u}(S, \boldsymbol{X}_S)\big) : \boldsymbol{u} \in \mathcal{H}_{\bar{\boldsymbol{K}}}, \|\boldsymbol{u}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \leq \tau\right\}. \tag{18}$$

Let $\mathcal{D}$ be a data distribution, and let $\hat{\mathcal{D}}_N = \{(\hat{S}_1, \boldsymbol{X}_{\hat{S}_1}, \hat{\boldsymbol{y}}_1), \ldots, (\hat{S}_N, \boldsymbol{X}_{\hat{S}_N}, \hat{\boldsymbol{y}}_N)\} \sim \mathcal{D}^N$ be a set of N independent samples drawn from $\mathcal{D}$.

We denote the empirical Rademacher complexity of $\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau)$ on $\hat{\mathcal{D}}_N$ as

$$R(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N) = \mathbb{E}_{\boldsymbol{\sigma}}\left[\sup_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)} \frac{1}{N}\sum_{i=1}^{N} \sigma_i \ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{u}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\right].$$

where $\sigma_1, \ldots, \sigma_N$ are independent uniform $\{\pm 1\}$-valued Rademacher random variables. Moreover, we denote

$$R_N(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) = \mathbb{E}_{\hat{\mathcal{D}}_N\sim\mathcal{D}^N}[R(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N)].$$

Definition 1 (Rademacher complexity on vector-valued function class)**.** The empirical Rademacher complexity of $\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)$ on $\hat{\mathcal{D}}_N$ is defined as

$$R(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N) = \mathbb{E}_{\boldsymbol{\sigma}}\left[\sup_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)} \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J} \sigma_{i,j}\boldsymbol{u}_j(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\right],$$

where $\sigma_{i,j}, i \in \{1, \ldots, N\}, j \in \{1, \ldots, J\}$ are independent Rademacher random variables. Moreover, the expected Rademacher complexity is

$$R_N(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) = \mathbb{E}_{\hat{\mathcal{D}}_N\sim\mathcal{D}^N}[R(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N)].$$

The proof relies on the following three lemmas.

Lemma 1. *Assume that*

$$\sup_{(\boldsymbol{u},\boldsymbol{y})} \ell(\boldsymbol{y}, \boldsymbol{u}(S, X_S)) - \inf_{(\boldsymbol{u},\boldsymbol{y})} \ell(\boldsymbol{y}, \boldsymbol{u}(S, \boldsymbol{X}_S)) \le M,$$

then with probability at least $1-\delta$*, for all* $\boldsymbol{u} \in \mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)$*:*

$$\mathbb{E}_{\mathcal{D}}\big[\ell(\boldsymbol{y}, \boldsymbol{u}(S, \boldsymbol{X}_S))\big] \le \frac{1}{N}\sum_{i=1}^{N} \ell\big(\hat{\boldsymbol{y}}_i, \boldsymbol{u}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big) + 2R_N(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) + M\sqrt{\frac{\ln(1/\delta)}{2N}}.$$

Proof. This Lemma follows from [Shalev-Shwartz and Ben-David, 2014, Theorem 26.5]. □

Lemma 2. *Consider* $\ell(\boldsymbol{u}, \boldsymbol{y})$ *that is* γ_2 *Lipschitz in* $\boldsymbol{u}$ *with respect to* L_2 *norm. Then the following contraction inequality exits*

$$R_N(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) \le \sqrt{2}\gamma_2 R_N(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}).$$

Proof. This lemma follows from Theorem 3 of [Maurer, 2016]. □

Lemma 3. *For a given dataset* $\hat{\mathcal{D}}_N$*, we have upper bound for its Rademacher complexity:*

$$R(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N) \le \frac{\tau}{N}\sqrt{\sum_{i=1}^{N}\sum_{j=1}^{J} \bar{\boldsymbol{K}}^{jj}\big((\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}), (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)},$$

$$R(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) \le \tau\sqrt{\frac{\mathbb{E}_{S\sim\mathcal{D}}\sum_{j=1}^{J} \bar{\boldsymbol{K}}^{jj}\big((S, \boldsymbol{X}_S), (S, \boldsymbol{X}_S)\big)}{N}}.$$

Proof.

$$R(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \hat{\mathcal{D}}_N) = \mathbb{E}\left[\sup_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)} \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J} \sigma_{i,j}\boldsymbol{u}_j(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\right].$$

Let $\boldsymbol{u} = \sum_{l=1}^{m} \bar{\boldsymbol{K}}\big(\cdot, (S_l, \boldsymbol{X}_{S_l})\big)\boldsymbol{\alpha}_l$, where $\boldsymbol{\alpha}_l \in \mathbb{R}^J$. Then

$$\begin{aligned}
&\mathbb{E}\left[\sup_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)} \frac{1}{N}\sum_{i=1}^{N} \sigma_{i,j}\boldsymbol{u}_j(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\right] \\
=&\mathbb{E}_{\boldsymbol{\sigma}} \sup_{\boldsymbol{\alpha}_1,\ldots,\boldsymbol{\alpha}_m\in\mathbb{R}^J} \left[\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sum_{l=1}^{m} \sigma_{i,j}\boldsymbol{e}_j^{\top}\bar{\boldsymbol{K}}\big((\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}), (S_l, \boldsymbol{X}_{S_l})\big)\boldsymbol{\alpha}_l\right] \\
=&\mathbb{E}_{\boldsymbol{\sigma}} \sup_{\boldsymbol{\alpha}_1,\ldots,\boldsymbol{\alpha}_m\in\mathbb{R}^J} \left[\left\langle \sum_{l=1}^{m}\bar{\boldsymbol{K}}\big(\cdot, (S_l, \boldsymbol{X}_{S_l})\big)\boldsymbol{\alpha}_l, \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\rangle_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right].
\end{aligned}$$

According to RKHS's inner product defined in (1), it can be expressed as

$$\mathbb{E}_{\boldsymbol{\sigma}} \sup_{\boldsymbol{\alpha}_1,\ldots,\boldsymbol{\alpha}_m\in\mathbb{R}^J} \left[\left\langle \sum_{l=1}^{m}\bar{\boldsymbol{K}}\big(\cdot, (S_l, \boldsymbol{X}_{S_l})\big)\boldsymbol{\alpha}_l, \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\rangle_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right].$$

By the Cauchy-Schwarz inequality in $\mathcal{H}_{\bar{\boldsymbol{K}}}$, for any function $\boldsymbol{f}, \boldsymbol{g} \in \mathcal{H}_{\bar{\boldsymbol{K}}}$,

$$\langle \boldsymbol{f}, \boldsymbol{g}\rangle_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \le \|\boldsymbol{f}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\|\boldsymbol{g}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}}.$$

Applying this inequality, we obtain

$$\begin{aligned}
&\mathbb{E}_{\boldsymbol{\sigma}} \sup_{\boldsymbol{\alpha}_1,\ldots,\boldsymbol{\alpha}_m\in\mathbb{R}^d} \left[\left\langle \sum_{l=1}^{m}\bar{\boldsymbol{K}}\big(\cdot, (S_l, \boldsymbol{X}_{S_l})\big)\boldsymbol{\alpha}_l, \frac{1}{N}\sum_{j=1}^{J}\sum_{i=1}^{N}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\rangle_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right] \\
&\le \mathbb{E}_{\boldsymbol{\sigma}} \sup_{\boldsymbol{u}\in\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau)} \left[\|\boldsymbol{u}\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}} \cdot \left\|\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right] \\
&\le \mathbb{E}_{\boldsymbol{\sigma}}\left[\tau \times \left\|\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right].
\end{aligned}$$

For any random variable X, $[\mathbb{E}X]^2 \le \mathbb{E}X^2$,

$$\begin{aligned}
&\mathbb{E}_{\boldsymbol{\sigma}}\left[\tau \times \left\|\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j\right\|_{\mathcal{H}_{\bar{\boldsymbol{K}}}}\right] \\
\le&\frac{\tau}{N}\sqrt{\mathbb{E}_{\boldsymbol{\sigma}}\left\langle \sum_{i=1}^{N}\sum_{j=1}^{J}\sigma_{i,j}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\boldsymbol{e}_j, \sum_{r=1}^{N}\sum_{s=1}^{J}\sigma_{r,s}\bar{\boldsymbol{K}}\big(\cdot, (\hat{S}_r, \boldsymbol{X}_{\hat{S}_r})\big)\boldsymbol{e}_s\right\rangle} \\
=&\frac{\tau}{N}\sqrt{\sum_{i=1}^{N}\sum_{j=1}^{J}\bar{\boldsymbol{K}}^{jj}\big((\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}), (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)}.
\end{aligned}$$

Finally, we can use the Jenson's inequality and the concavity of $\sqrt{\cdot}$ to deduce the second inequality. □

Lemma 4. *Given $\|\boldsymbol{u}\|_\infty \le R$, $\ell(\boldsymbol{y}, \boldsymbol{u}) \in [0, \log(J) + 2R]$ and is $\sqrt{2}$-Lipschitz continuous in $\boldsymbol{u}$ with respec to L_2 norm.*

Proof. This first result follows from the fact that $\boldsymbol{u} \in \mathbb{R}^J$ and $\|\boldsymbol{u}\|_\infty \leq R$. Then each exponential component in the softmax function ranges in $[e^{-R}, e^R]$. Thus, we have a minimal choice probability of $\frac{1}{J} \cdot e^{-2R}$. Recall that $\ell(\boldsymbol{y}, \boldsymbol{u}) = \ell(\boldsymbol{y}, \boldsymbol{p}(\boldsymbol{u})) = -\boldsymbol{y}^\top \log \boldsymbol{p}$. Also $\boldsymbol{y}$ is a choice vector. Then the loss is lower bounded by 0, achieved by $\boldsymbol{y} = \boldsymbol{p}$ and $\boldsymbol{y}$ is a one-hot vector. The maximum is obtained at $-\log\left(\frac{1}{J} \cdot e^{-2R}\right) = \log(J) + 2R$.

Recall the definition of $\ell(\boldsymbol{y}, \boldsymbol{u})$ in (10). Taking the gradient w.r.t. $\boldsymbol{u}$ yields

$$\nabla_{\boldsymbol{u}} \ell(\boldsymbol{y}, \boldsymbol{u}) = -\boldsymbol{y} + p(\boldsymbol{u}),$$

then $\| -\boldsymbol{y} + p(\boldsymbol{u})\|_2 \leq \sqrt{2}$ as desired. □

Using the above lemmas, we now prove Theorem 3.

Proof of Theorem 3. From Lemma 2 and 3, we obtain upper bound for $R_N(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D})$

$$R_N(\mathcal{L}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) \leq \sqrt{2}\gamma_2 R_N(\mathcal{H}_{\bar{\boldsymbol{K}}}(\tau), \mathcal{D}) \leq \sqrt{2}\gamma_2 \tau \sqrt{\frac{\mathbb{E}_{S\sim\mathcal{D}} \sum_{j=1}^J \bar{\boldsymbol{K}}^{jj}\big((S, \boldsymbol{X}_S), (S, \boldsymbol{X}_S)\big)}{N}}.$$

Also by Lemma 4, $\gamma = \sqrt{2}$ and $M = \log(J) + 2R$. Then substituting this upper bound into Lemma 1 yields the desired result. □

A.4 Proof of Proposition 2

Proof. Consider the scalar p−polynomial kernel: $\mathsf{k}(S, S') = (\boldsymbol{e}_S^\top \boldsymbol{e}_S')^p$. Its explicit feature map $\boldsymbol{\phi}_\mathsf{k}(S) \in \mathbb{R}^{N_p}$ contains all interaction terms up to order p:

$$\boldsymbol{\phi}_\mathsf{k}(S) = \left[\sqrt{\binom{p}{\boldsymbol{x}}} \prod_{j'=1}^J [\mathbb{1}(j \in S)]^{x_j}\right]_{\|\boldsymbol{x}\|_1 \leq p} \in \mathbb{R}^{N_p},$$

where $\|\boldsymbol{x}\|_1 = \sum_{j=1}^J x_j$ and $\binom{p}{\boldsymbol{x}} = \binom{p}{\boldsymbol{x}} = \frac{p!}{\prod_{j=1}^J x_j!}$ is the multinomial coefficient determined by the expansion of $(\boldsymbol{e}_S^\top \boldsymbol{e}_S)^\top$. The dimension $N_p = \binom{J+p}{p}$ counts all such monomials.

Each component correspnds to an interaction term $v_j^{k_1,\ldots,k_q}$ in (4) through:

$$\prod_{j'}^J \mathbb{1}(j' \in S)^{x_j} \begin{cases} 1 & \text{if } \{k : x_k > 0\} \subseteq S \\ 0 & \text{otherwise} \end{cases}.$$

To handle multiple dimensions $j \in \{1, \ldots, J\}$, we construct a block diagonal feature map via $\boldsymbol{\phi}(S) = \mathrm{diag}(\boldsymbol{\phi}_\mathsf{k}(S), \ldots, \boldsymbol{\phi}_\mathsf{k}(S)) \in \mathbb{R}^{N_p J \times J}$. Namely, for each dimension $j \in \{1, \ldots, J\}$, $\boldsymbol{\phi}_j(S) \in \mathbb{R}^{N_p J}$ can be expressed as

$$\boldsymbol{\phi}_j(S) = \begin{bmatrix} \boldsymbol{0}_{N_p(j-1)} \\ \boldsymbol{\phi}_\mathsf{k}(S) \\ \boldsymbol{0}_{N_p(J-j)} \end{bmatrix}.$$

The induced kernel becomes $\boldsymbol{K}(S, S') = (\boldsymbol{e}_S^\top \boldsymbol{e}_{S'})^p \cdot \boldsymbol{I}_J$. Finally, we can modify the feature map using the masking mechanism in (2) to enforce $u_j(S) = 0, j \notin S$. This yields the final feature map and the corresponding kernel $\bar{\boldsymbol{K}}$

$$\bar{\boldsymbol{K}} = [\langle \bar{\boldsymbol{\phi}}_i(S), \bar{\boldsymbol{\phi}}_j(S)\rangle]_{ij}^J, \quad \text{with } \bar{\boldsymbol{\phi}}_j(S) = \mathbb{1}(j \in S)\boldsymbol{\phi}_j(S).$$

□

B Random Feature Procedure

According to (2), the feature map $\bar{\boldsymbol{\phi}} =: [\bar{\phi}_j]_{j=1}^J$ must satisfy the following constraints to ensure a consistent domain for all entries of $\boldsymbol{u}$, namely,

$$\bar{\phi}_j(S, \boldsymbol{X}_S; w) = \mathbb{1}(j \in S) \cdot \phi_j(S, \boldsymbol{X}_S; w), \quad \forall j \in \{1, \ldots, J\}. \tag{19}$$

For the remaining section we describe the procedure using random features to approximate the unmasked kernels, following from [Sinha and Duchi, 2016]. For masked kernels, simply add the mask to the kernels involved in the optimization problems.

Given the training set $\hat{\mathcal{D}}_N$, we convert the choices $\hat{\boldsymbol{y}}_i$ into similarity vectors $\hat{\mathbf{s}}_i$ by replacing every 0 in $\hat{\boldsymbol{y}}_i$ with -1,

$$\hat{s}_{i,j} = \begin{cases} 1 & \text{if } \hat{y}_{i,j} = 1 \\ -1 & \text{if } \hat{y}_{i,j} = 0 \end{cases}.$$

We consider the following kernel alignment problem to find the optimal Q

$$\max_{Q \in \mathcal{P}} \sum_{i,j} \hat{\mathbf{s}}_i^\top \boldsymbol{K}_Q\big((\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}), (\hat{S}_i, \boldsymbol{X}_{\hat{S}_i})\big)\hat{\mathbf{s}}_j, \tag{20}$$

where $\mathcal{P}$ is the set of the divergence measures on the space of probability distributions. For a convex function f with $f(1) = 0$, the f-divergence between distributions P and Q is $D_f(P\|Q) = \int f\left(\frac{dP}{dQ}\right) dQ$. Then for a base distribution P_0, we consider collections $\mathcal{P} := \{Q : D_f(Q\|P_0) \leq \rho\}$ where $\rho > 0$ is a specified constant. $f(t) = t^k - 1$ for $k \geq 2$.

Recall the discrete approximation

$$\sum_{m=1}^{N_w} q_m \boldsymbol{\phi}(S, X_S; w_m)\boldsymbol{\phi}(S', X_{S'}; w_m)^\top,$$

where $\{w_m\}_m$ are pre-sampled. Substituting it into (20), we obtain the approximated kernel alignment problem with $[-1, 1]$ similarity score:

$$\max_{\boldsymbol{q} \in \Delta_{N_w}} \sum_{i,j} \sum_{m=1}^{N_w} q_m \hat{\mathbf{s}}_i^\top \boldsymbol{\phi}(\hat{S}_i, X_{\hat{S}_i}; w_m)\boldsymbol{\phi}(\hat{S}_j, X_{\hat{S}_j}; w_m)^\top \hat{\mathbf{s}}_j. \tag{21}$$

We can show that the alignment problem (13) can be efficiently solved. Similar argument applies to the problem (21) by replacing y with s.

Given a sample w, we define the feature map for dataset $\hat{\mathcal{D}}_N$ as

$$\boldsymbol{\phi}(\hat{\mathcal{D}}_N, w) = \begin{bmatrix} \boldsymbol{\phi}(\hat{S}_1, \boldsymbol{X}_{\hat{S}_1}; w)^\top & \boldsymbol{\phi}(\hat{S}_2, \boldsymbol{X}_{\hat{S}_2}; w)^\top & \cdots & \boldsymbol{\phi}(\hat{S}_N, \boldsymbol{X}_{\hat{S}_N}; w)^\top \end{bmatrix} \in \mathbb{R}^{1 \times NJ}. \tag{22}$$

We further construct the feature matrix for N_w samples as

$$\Phi(\hat{\mathcal{D}}_N, \mathcal{W}_{N_w}) = \begin{bmatrix} \boldsymbol{\phi}(\hat{\mathcal{D}}_N, w_1) \\ \vdots \\ \boldsymbol{\phi}(\hat{\mathcal{D}}_N, w_{N_w}) \end{bmatrix} \in \mathbb{R}^{N_w \times (NJ)}, \tag{23}$$

where $\mathcal{W}_{N_w}$ denotes the set of N_w samples drawn from $\mathcal{W}$. Finally, we concatenate all the actual choice vectors. Define

$$\hat{Y} = \begin{bmatrix} \hat{\boldsymbol{y}}_1 \\ \vdots \\ \hat{\boldsymbol{y}}_N \end{bmatrix} \in \mathbb{R}^{NJ}.$$

The problem (13) can be expressed as

$$\begin{aligned} &\max_{Q\in\mathcal{P}} \sum_{i,j} \sum_{m=1}^{N_w} q_m \hat{\boldsymbol{y}}_i^\top \boldsymbol{\phi}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}; w_m) \boldsymbol{\phi}(\hat{S}_j, \boldsymbol{X}_{\hat{S}_j}; w_m)^\top \hat{\boldsymbol{y}}_j \\ =&\max_{Q\in\mathcal{P}} \sum_{m=1}^{N_m} q_m \sum_{i,j} \left(\boldsymbol{\phi}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}; w_m)^\top \hat{\boldsymbol{y}}_i\right)^\top \left(\boldsymbol{\phi}(\hat{S}_j, \boldsymbol{X}_{\hat{S}_j}; w_m) \hat{\boldsymbol{y}}_j\right) \\ =&\max_{Q\in\mathcal{P}} \sum_{m=1}^{N_m} q_m \left(\sum_{i=1}^{N} \boldsymbol{\phi}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}; w_m)^\top \hat{\boldsymbol{y}}_i\right)^2. \end{aligned}$$

Then we have

$$\max_{Q\in\mathcal{P}} \sum_{m=1}^{N_w} q_m \left(\sum_{i=1}^{N} \boldsymbol{\phi}(\hat{S}_i, \boldsymbol{X}_{\hat{S}_i}; w_m)^\top \hat{\boldsymbol{y}}_i\right)^2 = \boldsymbol{q}^\top \boldsymbol{V}.$$

where $\boldsymbol{q} = [q_1, \cdots, q_{N_w}]^\top \in \mathbb{R}^{N_w}$, $\boldsymbol{V} = \left(\Phi(\hat{\mathcal{D}}_N, \mathcal{W}_{N_w})\hat{Y}\right) \odot \left(\Phi(\hat{\mathcal{D}}_N, \mathcal{W}_{N_w})\hat{Y}\right) \in \mathbb{R}^{N_w}$, and $\odot$ denotes the Hadamard product.

Consider $\lambda_J \geq 0$ for the constraint $D_f(Q\|P_0) \leq \rho$, we can write the partial Lagrangian form of problem (13):

$$L(\boldsymbol{q}, \lambda) = \boldsymbol{q}^\top \boldsymbol{V} - \lambda_J D_f(Q\|P_0).$$

The dual function is $g(\lambda_J) = \sup_{\boldsymbol{q}\in\Delta} L(\boldsymbol{q}, \lambda_J)$, where $\Delta := \{\boldsymbol{q} \in \mathbb{R}_+^{N_w} : \boldsymbol{q}^\top \mathbf{1} = 1\}$. For $f(t) = t^k - 1$, we solve

$$\max_{\boldsymbol{q}\in\Delta} \boldsymbol{q}^\top \boldsymbol{V} - \lambda_J \frac{1}{N_w} \sum_{i=1}^{N_w} (N_w q_i)^k. \tag{24}$$

(24) has the solution of the form $q_i = \left[V_i/\lambda_J N_w^{k-1} + \xi\right]^{1/(k-1)}$, where ξ is chosen to ensure that $\boldsymbol{q} \in \Delta$, which can be quickly obtained using bisect.

C Neural Tangent Kernel

We prove Theorem 2 as follows. To ease notation, we set $x = (S, \boldsymbol{X}_S)$. Define

$$v_H^{NTK}(x; \boldsymbol{\theta}) = \tilde{v}_H^{NTK}(x) + \frac{1}{\sqrt{H}} \sum_{h=1}^{H} \phi(x; \tilde{\Theta}^h)(w^h - \tilde{w}^h) + \frac{1}{\sqrt{H}} \sum_{h=1}^{H} (\Theta^h - \tilde{\Theta}^h)^\top \nabla_\Theta \phi(x; \tilde{\Theta}^h) \tilde{w}^h.$$

The following results are standard from NTK theory (e.g., see [Zhang, 2023, Chapter 11]).

Lemma 5. *Suppose $\tilde{\boldsymbol{\theta}}$ is initialized according to a Gaussian $\mathcal{N} \sim N(0, \sigma^2 I)$. Then there exist $A_0, r_0 > 0$ such that for sufficiently large H, with probability at least $1 - \delta$, the following holds for all $\boldsymbol{\theta}$ with*

$\|\boldsymbol{\theta} - \tilde{\boldsymbol{\theta}}\|_\infty \le r_0$ *and all* $x \in \hat{\mathcal{D}}$*:*

$$\begin{aligned}
&\|\tilde{\boldsymbol{v}}_H^{NTK}(x)\| \le A_0, \quad \|\nabla_{\boldsymbol{\theta}} \boldsymbol{v}_H^{NTK}(x;\boldsymbol{\theta})\|_2 + \|\nabla_{\boldsymbol{\theta}} \boldsymbol{v}(x;\boldsymbol{\theta})\|_2 \le A_0,\\
&\|\boldsymbol{v}_H^{NTK}(x;\boldsymbol{\theta}) - \boldsymbol{v}(x;\boldsymbol{\theta})\|_2 \le \epsilon, \quad \|\nabla_{\boldsymbol{\theta}} \boldsymbol{v}_H^{NTK}(x;\boldsymbol{\theta}) - \nabla_{\boldsymbol{\theta}} \boldsymbol{v}(x;\boldsymbol{\theta})\|_2 \le \epsilon,\\
&\|\nabla \boldsymbol{v}_H^{NTK}(x;\boldsymbol{\theta})\|_\infty + \|\nabla \boldsymbol{v}(x;\boldsymbol{\theta})\|_\infty = O(H^{-1/4}).
\end{aligned}$$

Set

$$\boldsymbol{K}_H^{NTK}(X_S, X'_{S'}) := \frac{1}{H}\sum_{h=1}^{H} \phi(X_S;\tilde{\Theta}^h)\phi(X'_{S'};\tilde{\Theta}^h)^\top + \frac{1}{H}\sum_{h=1}^{H} \nabla_\Theta \phi(X_S;\tilde{\Theta}^h)\tilde{w}^h \big(\nabla_\Theta \phi(X_S;\tilde{\Theta}^h)\tilde{w}^h\big)^\top.$$

Lemma 6. *For every* x, x'*, as* $H \to \infty$*, it holds that*

$$\boldsymbol{K}_H^{NTK}(x,x') \xrightarrow{p.} \boldsymbol{K}^{NTK}(x,x').$$

Lemma 7. *Assume* $\hat{\mathcal{D}}$ *consists of distinct points and* $\boldsymbol{K}^{NTK}$ *is universal. For any function* $\boldsymbol{v}$*, there exists a function* $\boldsymbol{v}_H^{NTK}$ *and a parameter* $\boldsymbol{\theta}$ *such that* $\|\boldsymbol{\theta} - \tilde{\boldsymbol{\theta}}\|_\infty = O(H^{-1/4})$*,* $\|\boldsymbol{\theta} - \tilde{\boldsymbol{\theta}}\|_2 = O(1)$ *and*

$$\|\boldsymbol{v}_H^{NTK}(\hat{x}^i;\boldsymbol{\theta}) - \boldsymbol{v}(\hat{x}^i)\| \le \epsilon, \quad \forall i = 1,\dots,n.$$

Proof of Theorem 2. We set $\boldsymbol{\theta}_0 = \tilde{\boldsymbol{\theta}}$. We prove by induction that all iterates are in the NTK regime. The base case $k = 0$ holds trivially from Lemma 5. Now suppose $\boldsymbol{\theta}_k$ satisfies $\|\boldsymbol{\theta}_k - \tilde{\boldsymbol{\theta}}\|_\infty = O(H^{-1/4})$. Using the SGD update and Lemma 5, we have that

$$\|\boldsymbol{\theta}_{k+1} - \tilde{\boldsymbol{\theta}}\|_\infty \le \|\boldsymbol{\theta}_k - \tilde{\boldsymbol{\theta}}\|_\infty + \eta\|\hat{g}_k\| = O(H^{-1/4}).$$

Denote by $\mathcal{F}_k$ the filtration generated by $\{z_{k'}\}_{k'=1}^k$. Thus conditional on $\mathcal{F}_k$, we have

$$\mathbb{E}[\|\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}\|_2^2 \mid \mathcal{F}_k] \le \|\boldsymbol{\theta}_k - \boldsymbol{\theta}\|_2^2 - 2\eta\mathbb{E}[\hat{g}_k(z_k) \mid \mathcal{F}_k]^\top(\boldsymbol{\theta}_k - \boldsymbol{\theta}) + \eta^2\mathbb{E}[\|\hat{g}_k(z_k)\|_2^2 \mid \mathcal{F}_k].$$

Using Lemma 7, for sufficiently large H, with high probability,

$$\|\nabla L(\boldsymbol{v}_H^{NTK}(\cdot;\boldsymbol{\theta}_k)) - \mathbb{E}[\hat{g}_k(z_k) \mid \mathcal{F}_k]\|_2 \le \epsilon.$$

It follows that

$$\begin{aligned}
\mathbb{E}[\hat{g}_k(z_k)]^\top(\boldsymbol{\theta}_k - \boldsymbol{\theta}) &\ge \nabla L(\boldsymbol{v}_H^{NTK}(\cdot;\boldsymbol{\theta}_k))^\top(\boldsymbol{\theta}_k - \boldsymbol{\theta}) - O(\epsilon)\\
&\ge L(\boldsymbol{v}_H^{NTK}(\cdot;\boldsymbol{\theta}_k)) - L(\boldsymbol{v}_H^{NTK}(\cdot;\boldsymbol{\theta})) - O(\epsilon)\\
&\ge L(\boldsymbol{v}(\cdot;\boldsymbol{\theta}_k)) - L(\boldsymbol{v}(\cdot;\boldsymbol{\theta})) - O(\epsilon),
\end{aligned}$$

where the last inequality follows from Lemma 5. Using telescope sum as in standard SGD analysis yields the result. □

D Experiment Details for Synthetic data

Table 7 summarizes the predefined base utility values and first-order halo interactions used in the data generation. Building on the first-order interaction described above, the parameters of second-order interactions specified in Table 8. For RKHS-based methods, when $p = 1$, we employed the linear kernel (equivalent to the first-order polynomial kernel), which captures first-order interactions.

Base Utility	Item	1	2	3	4	5
-0.925	1	0.00	−0.11	0.18	−0.15	0.28
-0.425	2	0.17	0.00	−0.19	−0.16	−0.18
-2.644	3	−0.20	−0.07	0.00	0.08	−0.14
0.145	4	−0.03	−0.06	0.36	0.00	0.33
-0.121	5	0.08	−0.10	0.04	−0.03	0.00

Table 7. Ground Truth: Base Utility Values and First-order Interactions

Table 8. Ground Truth: Second-order Interactions

Item Pair		Interaction				
i	j	1	2	3	4	5
1	2	0.000	0.000	0.008	0.011	−0.025
1	3	0.000	−0.020	0.000	0.074	0.139
1	4	0.000	0.029	0.052	0.000	−0.002
1	5	0.000	0.038	−0.037	0.076	0.000
2	3	0.133	0.000	0.000	0.029	−0.057
2	4	0.075	0.000	0.063	0.000	0.023
2	5	−0.090	0.000	−0.097	0.005	0.000
3	4	−0.009	−0.140	0.000	0.000	−0.137
3	5	0.021	−0.035	0.000	0.093	0.000
4	5	0.046	0.049	0.075	0.000	0.000

For $p = 3$, we generate the ground truth parameters as follows. Base utilities are randomly sampled from $\mathcal{N}(0, 1)$, first-order interactions from $\mathcal{N}(0, 0.5)$, second-order interactions from $\mathcal{N}(0, 0.25)$, and third-order interaction from $\mathcal{N}(0, 0.125)$. The parameters are fixed once generated; due to the large quantities of the interaction coefficients we omit the exact values from this paper.

E Real Data Proprocessing

E.1 Dataset without Feature

During the data preprocessing phase, we excluded transactions where the purchased room type was not available as part of the declared offered set. Furthermore, we retain only the offered choice

sets $\hat{S}_i$ and corresponding actual choices for $i \in \{1, \ldots, N\}$. Both are then converted into one-hot encodings: the offered choice set is represented as $\boldsymbol{e}_{\hat{S}_i} \in \{0,1\}^J$, and the actual choice is encoded as $\hat{\boldsymbol{y}}_i \in \{0,1\}^J$, where J denotes the total number of room types, including the no-purchase option.

E.2 Dataset with Feature

The data cleaning procedure follows the methodology described in Aouad and Désir [2023]. For each transaction in both datasets, we construct a feature matrix of size $J \times d$, where J denotes the maximal cardinality across all observations, and d is the dimensionality of the concatenated feature vector. Each row of the matrix corresponds to an item in the choice set, represented by the concatenation of item-specific and customer-specific features.

If the cardinality of the offered choice set is smaller than J, the remaining rows are padded with zeros to maintain a consistent matrix size. In datasets that include a no-purchase option, the last row of the padded matrix (i.e., the final zero row) represents the no-purchase alternative.

F Experiment Details for Fixed Kernel

F.1 Procedure Details

Experimental Hardware Setup All experiments were conducted on a high-performance computational platform equipped with an AMD EPYC 7302 16-core processor, 251 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU. The operating system used was Ubuntu 20.04.4 LTS (Focal Fossa). The implementation was carried out in Python 3.9, utilizing the PyTorch 2.5.1 framework for model training and evaluation. This configuration ensured efficient handling of the computational demands associated with kernel matrix precomputation and optimization during the training process.

Data Splitting The training process begins with data splitting. Each hotel dataset is randomly split into training and test sets in an $8:2$ ratio. The training set is used to optimize the model, while the test set evaluates the model's performance on out-of-sample data. For the feature-based RKHS choice model, we employ a 10-fold cross-validation to ensure robust evaluation. In this setup, the Swissmetro dataset is divided into 10 folds, where 8 folds are used for training, 1 fold is used for validation, and the remaining is for test. The test set was never seen during the training, which enabled us to assess the model's out-of-sample performance.

Selecting the Unmasked Kernel and Masking In both datasets, the input data consists of offered choice sets and the corresponding actual choices. The process begins with selecting a base kernel $\boldsymbol{K}$ to measure the similarity between the offered choice set. This kernel operates on either the one-hot encoded vectors(featureless case) or the feature matrices (feature-based case) as detailed in Section 3. Next, a masking procedure is applied to the unmasked kernel $\boldsymbol{K}$, as described in (2), to derive the final kernel $\bar{\boldsymbol{K}}$.

Precompute Kernel Values Following the kernel selection and masking, we precompute the kernel values for all samples. The resulting tensor, $\mathcal{K} \in \mathbb{R}^{N \times N \times J \times J}$, encodes kernel values for every pair of samples. Specifically, each entry $\mathcal{K}[i,j]$ represents a $J \times J$ matrix corresponding to the kernel value between $\hat{S}_i$ and $\hat{S}_j$. This step ensures that the kernel matrix is readily available for subsequent model training and evaluation, reducing computation time during the learning phase. During training, we optimize the set of parameters $\boldsymbol{\alpha}$ as described in (12).

Early Stopping Creteria To prevent overfitting and ensure convergence, the training process is terminated based on the following stopping criterion: the gradient norm is monitored during training, and if the gradient norm becomes smaller than a predefined value ε, the training process is

stopped. Additionally, for the Swissmetro datasets, we also monitor the validation loss during training. If the number of epochs without improvement in the validation loss exceeds a specified patience, the training is stopped as well.

F.2 Hyperparameters and Additional Experiment Results

Table 9 outlined the foundational training parameters and setups and the kernel-specific configurations are detailed in Table 10.

Hyperparameter	**Hotel Dataset**	**Swissmetro Dataset**
Regularization Parameter (λ)	1×10^{-6}	5×10^{-5}
Gradient Norm Threshold (ε)	1×10^{-4}	5×10^{-3}
Initial Learning Rate	0.01	0.01
Patience	–	15
Optimizer	Adam	L-BFGS

Table 9. Fixed Kernel Training Configuration

Dataset	Kernel	Parameter	Value
Hotel	Linear	–	–
	Polynomial	c	1
		f	2
	Gaussian	ℓ	1
	Matérn	ℓ	1
		ν	0.5
Swissmetro	Gaussian	σ	1/5
		ℓ	1/3.6
	Matérn	σ	1/5
		ℓ	1/3.6
		ν	0.5/1.5/2.5

Table 10. Kernel Hyperparameters for Hotel and Swissmetro Datasets

To evaluate the performance of different kernel functions under regularization, Table 11 compares various kernels across the Hotel dataset. The table presents key performance metrics, including RMSE, Negative Log-Likelihood, and runtime, under a fixed regularization parameter (($\lambda = 1 \cdot 10^{-6}$) to illustrate the effects of different kernel configurations on model performance. Table 12 presents the kernel comparison for the Hotel dataset without regularization $\lambda = 0$.

Hotel	Kernel	RMSE		Negative Log-Likelihood		Runtime (s)
		Test	Train	Test	Train	
1	Linear	0.22190	0.21474	0.78729	0.79879	329.49
	Polynomial	0.22201	0.21477	0.78819	0.79804	**238.77**
	Sigmoid	0.22376	0.21655	0.81737	0.83172	338.99
	Gaussian	0.22183	0.21464	**0.78621**	0.79463	338.90
	Matérn	**0.22172**	**0.21449**	0.78388	**0.78820**	258.27
2	Linear	**0.20878**	0.20780	**0.75517**	0.75335	70.02
	Polynomial	0.20958	0.20836	0.76081	0.75118	65.61
	Sigmoid	0.20956	0.20870	0.77465	0.77509	55.60
	Gaussian	0.20908	0.20695	0.76538	0.71768	67.36
	Matérn	0.20918	**0.20678**	0.76147	**0.70475**	**48.21**
3	Linear	0.20746	0.20647	**0.71387**	0.71394	104.32
	Polynomial	0.20819	0.20670	0.71675	0.71334	106.42
	Sigmoid	0.20857	0.20755	0.74923	0.74662	**79.95**
	Gaussian	0.20747	0.20624	0.71597	0.70628	106.34
	Matérn	**0.20743**	**0.20620**	0.71682	**0.70545**	106.07
4	Linear	**0.28169**	0.27646	0.71850	**0.70051**	46.82
	Polynomial	0.28236	**0.27660**	0.72257	0.70055	62.51
	Sigmoid	0.28232	0.27738	0.72411	0.70676	29.58
	Gaussian	0.28162	0.27657	0.71822	0.70266	30.39
	Matérn	0.28171	0.27678	**0.71705**	0.70523	**22.39**
5	Linear	0.23908	0.23806	0.79070	0.75370	68.66
	Polynomial	0.23917	0.23787	0.79226	0.74820	63.86
	Sigmoid	0.23938	0.23846	0.79402	0.76219	57.05
	Gaussian	0.23890	0.23765	0.78817	0.747584	42.29
	Matérn	**0.23890**	**0.23759**	**0.78785**	**0.74586**	**38.18**

Table 11. Comparison of Kernel Functions Across Hotels with regularization ($\lambda = 1 \cdot 10^{-6}$)

Hotel	Kernel	RMSE		Negative Log-Likelihood		Runtime (s)
		Test	Train	Test	Train	
1	Linear	0.22185	0.21534	0.78564	0.80108	187.13
	Polynomial	0.22219	0.21554	0.788900	0.80254	86.44
	Sigmoid	0.22364	0.21702	0.81631	0.83262	103.65
	Gaussian	0.22178	0.21516	**0.78447**	0.79304	58.33
	Matérn	**0.22172**	**0.21511**	0.78528	**0.79183**	**34.45**
2	Linear	**0.20900**	0.20797	**0.75787**	0.75662	46.87
	Polynomial	0.21015	0.20826	0.76024	0.75728	49.79
	Sigmoid	0.20959	0.20870	0.77342	0.77350	**14.54**
	Gaussian	0.20938	0.20736	0.76130	0.73184	39.62
	Matérn	0.20915	**0.20717**	0.76183	**0.72399**	42.02
3	Linear	**0.20772**	0.20654	0.71529	0.71390	60.92
	Polynomial	0.20858	0.20664	0.71689	0.71348	61.41
	Sigmoid	0.20862	0.20752	0.73424	0.73170	59.05
	Gaussian	0.20745	0.20626	**0.71542**	0.70726	**55.94**
	Matérn	0.20741	**0.20621**	0.71643	**0.70598**	58.74
4	Linear	**0.28179**	0.27636	**0.72143**	0.69907	40.58
	Polynomial	0.28261	0.27647	0.72281	0.69955	44.81
	Sigmoid	0.28244	0.27711	0.73001	0.70315	**13.67**
	Gaussian	0.28207	0.27619	0.72871	0.69612	20.45
	Matérn	0.28206	**0.27617**	0.72962	**0.69520**	22.97
5	Linear	0.23921	0.23803	0.79400	0.75249	46.29
	Polynomial	0.23970	0.23806	0.79433	0.75191	50.80
	Sigmoid	0.23962	0.23855	0.79923	0.76287	**13.40**
	Gaussian	0.23900	0.23760	**0.79024**	0.74560	26.11
	Matérn	**0.23896**	**0.23754**	0.79042	**0.74393**	28.43

Table 12. Comparison of Kernel Functions Across Hotels without regularization ($\lambda = 0$)

Hotel	Metric	Kernel				
		Linear	Poly	Sigmoid	Gaussian	Matérn
1	RMSE	1.71e-05	8.85e-06	6.99e-06	6.65e-06	8.04e-06
	NLL	5.74e-04	1.37e-04	1.69e-04	1.51e-04	1.58e-04
2	RMSE	3.38e-06	1.98e-05	2.03e-06	1.37e-05	6.65e-06
	NLL	8.97e-05	1.43e-04	6.04e-05	1.38e-04	1.28e-04
3	RMSE	4.40e-06	1.11e-05	4.86e-06	3.96e-05	5.64e-05
	NLL	2.52e-04	1.77e-04	1.61e-04	3.08e-04	3.94e-04
4	RMSE	1.00e-05	1.10e-05	3.56e-06	4.30e-06	5.64e-06
	NLL	1.56e-04	2.19e-04	1.41e-04	1.24e-04	1.80e-04
5	RMSE	1.40e-05	2.72e-05	4.27e-06	1.52e-05	9.26e-05
	NLL	1.79e-04	2.66e-04	1.31e-04	1.58e-04	1.79e-04

Table 13. Standard Deviation of Test RMSE and NLL for Different Kernels Across Hotels

Each sample in Table 11 and Table 12 was repeated 20 times. To further assess the differences between kernel functions, we performed a t-test. The standard deviation of Test RMSE and NLL for different kernels are presented Table 13. The differences between each kernel's performance were found to be statistically significant ($p < 0.05$).

Table 14 presents the predictive performance of different models on the Swissmetro dataset, comparing both in-sample and out-of-sample results. Comparison of different kernels and parameters are detailed in Table 15.

G Experiment Details for Random Feature

This section explain how we implement the Random Feature model. After using the initial distribution Q to construct the base feature map, we apply the masking procedure as described in (19) to obtain feature map $\boldsymbol{\phi}(S, X_S; w)$ for each random sample. Following this, we can construct the feature matrix for the total samples and dataset according as described in (22) and (23). The kernel alignment problem (20) is solved to optimize the distribution and the final learning problem finally (14) is subsequently trained. Model training is performed using Adam optimizer, with gradient norm thresholding and validation patience incorporated to prevent overfitting.

Model	In-sample		Out-of-sample	
	NLL	Accuracy	NLL	Accuracy
MNL	0.837	0.624	0.837	0.623
TasteNet	0.385	0.846	0.558	0.790
RUMnet	0.324	0.874	0.546	0.797
Vanilla Neural Network	0.434	0.823	0.581	0.759
Fixed Kernel(0.5-Matérn)	**0.293**	**0.900**	**0.381**	**0.845**
Fixed Kernel(1.5-Matérn)	0.391	0.832	0.468	0.783
Fixed Kernel(2.5-Matérn)	0.406	0.821	0.482	0.821
Fixed Kernel(Gaussian)	0.421	0.811	0.498	0.763

Table 14. Predictive performance on Swissmetro dataset (in-sample vs. out-of-sample)

Kernel type	Kernel Parameters	Train NLL	Val NLL	Test NLL	Train Accuracy	Val Accuracy	Test Accuracy	Train Kernel Precompute Time	Test Kernel Precompute Time	Fit Time
Gaussian	$(\sigma, \ell) = (1, 1)$	0.540	0.620	0.600	0.762	0.711	0.721	0.011	0.001	47.400
	$(\sigma, \ell) = (1, 3.6)$	0.713	0.720	0.715	0.672	0.668	0.669	0.011	0.001	64.031
	$(\sigma, \ell) = (5, 1)$	0.433	0.605	0.558	0.794	0.731	0.740	0.011	0.001	37.368
	$(\sigma, \ell) = (5, 3.6)$	0.593	0.625	0.629	0.744	0.726	0.728	0.011	0.001	137.355
Matérn	$(\sigma, \ell, \nu) = (1, 1, 0.5)$	0.503	0.603	0.580	0.789	0.719	0.731	56.348	7.039	41.334
	$(\sigma, \ell, \nu) = (1, 1, 1.5)$	0.524	0.612	0.588	0.770	0.711	0.723	57.987	7.220	48.488
	$(\sigma, \ell, \nu) = (1, 1, 2.5)$	0.528	0.614	0.591	0.768	0.711	0.721	121.066	15.102	46.163
	$(\sigma, \ell, \nu) = (1, 3.6, 0.5)$	0.563	0.627	0.613	0.765	0.718	0.726	56.324	7.048	53.738
	$(\sigma, \ell, \nu) = (1, 3.6, 1.5)$	0.645	0.673	0.669	0.709	0.687	0.689	57.967	7.222	53.426
	$(\sigma, \ell, \nu) = (1, 3.6, 2.5)$	0.676	0.693	0.690	0.688	0.677	0.681	121.065	15.131	60.431
	$(\sigma, \ell, \nu) = (5, 1, 0.5)$	0.308	0.557	0.517	0.877	0.764	0.778	56.271	7.036	56.229
	$(\sigma, \ell, \nu) = (5, 1, 1.5)$	0.416	0.600	0.558	0.801	0.736	0.741	57.957	7.227	30.004
	$(\sigma, \ell, \nu) = (5, 1, 2.5)$	0.423	0.605	0.560	0.798	0.735	0.740	121.007	15.138	37.223
	$(\sigma, \ell, \nu) = (5, 3.6, 0.5)$	0.381	0.547	0.518	0.833	0.760	0.764	56.365	7.032	61.876
	$(\sigma, \ell, \nu) = (5, 3.6, 1.5)$	0.435	0.536	0.521	0.818	0.763	0.770	57.956	7.227	101.143
	$(\sigma, \ell, \nu) = (5, 3.6, 2.5)$	0.479	0.554	0.552	0.800	0.753	0.760	121.037	15.121	113.217

Table 15. Comparison of Fixed Kernel Types and Parameters

G.1 Hyperparameters and Additional Experiment Results

Hyperparameter	Hotel Dataset	Swissmetro Dataset
Regularization Parameter (λ)	1×10^{-10}	1×10^{-10}
Gradient Norm Threshold (ε)	5×10^{-5}	1×10^{-6}
Initial Learning Rate	0.01	0.01
Patience	–	10
Optimizer	Adam	Adam

Table 16. Random Feature Training Configuration

Basic training configuration setup for random feature experiments are summarized in Table 16. And the Gaussain random feature are detailed in Table 17.

Parameter	Hotel	Swissmetro
σ	1	5
ℓ	1	3.6

Table 17. Gaussian Kernel Random Feature Parameters

Hotel	RMSE		Negative Log-Likelihood		Alignment Time (s)	Train time (s)
	Test	Train	Test	Train		
1	0.22222	0.21563	0.79369	0.80626	0.91	206.13
2	0.20988	0.20782	0.77641	0.75711	0.64	195.94
3	0.20789	0.20666	0.72603	0.72042	1.10	274.09
4	0.28161	0.27639	0.71758	0.70042	0.63	23.16
5	0.23888	0.23775	0.78859	0.75090	0.62	50.69

Table 18. Hotel Dataset Random Feature

Table 18 reports the performance of the model on the Hotel dataset with randomly initialized features. The evaluation metrics include both RMSE and Negative Log-Likelihood for both the test and train sets, along with the alignment time and total training time in seconds.

H Experiment Details for Neural Tangent Kernel

We implement a two-layer Neural Tangent Kernel with 1024 units for the feature-free Hotel dataset. Table 19 and 20 present the model's performance across the five hotels with different activation functions and optimizers. The model parameters are randomly initialized from a Gaussian distribution $\mathcal{N}(0, 0.01)$.

Table 19. Hotel Dataset 2-layer Neural Tangent Kernel (Adam 1024 units)

Hotel	Activation	RMSE		Negative Log-Likelihood		Runtime (s)
		Test	Train	Test	Train	
1	**ReLU**	0.22158	0.21497	0.78127	0.78847	0.64
	Sigmoid	0.22169	0.21525	0.78778	0.80357	2.12
2	ReLU	0.20998	0.20653	0.83790	0.69518	1.25
	Sigmoid	0.20881	0.20764	0.77009	0.74932	2.63
3	ReLU	0.20749	0.20616	0.71498	0.70570	0.49
	Sigmoid	0.20741	0.20636	0.71888	0.71543	2.63
4	**ReLU**	0.28221	0.27601	0.75501	0.69234	0.49
	Sigmoid	0.28231	0.27615	0.75177	0.69725	2.69
5	**ReLU**	0.23909	0.23721	0.80484	0.73526	0.62
	Sigmoid	0.23919	0.23762	0.80083	0.75150	2.01

Table 20. Hotel Dataset 2-layer Neural Tangent Kernel (SGD 1024 units)

Hotel	RMSE		Negative Log-Likelihood		Runtime (s)
	Test	Train	Test	Train	
1	0.22174	0.21518	0.79040	0.80299	24.99
2	0.20868	0.20730	0.76445	0.73859	35.65
3	0.20746	0.20632	0.72106	0.71577	14.13
4	0.28214	0.27610	0.74318	0.69701	9.02
5	0.23907	0.23749	0.79853	0.74730	13.60

For Swissmetro dataset, our neural tangent kernel with attention implementation employs a 1,000-head attention architecture optimized for the feature-rich Swissmetro dataset. Parameters are initialized from $\mathcal{N}(0, 0.01)$ to balance expressivity with gradient stability, while Adam optimization (learning rate = 0.01, no weight decay) ensures efficient traversal of the high-dimensional parameter space. A gradient noise injection mechanism ($\tau = 10^{-5}$) enhances convergence robustness against saddle points – particularly crucial given the dataset's mixed discrete-continuous

feature composition. This configuration preserves the NTK's theoretical convergence guarantees while accommodating the practical challenges of travel choice modeling through three key mechanisms: multi-head attention for feature interaction capture, controlled initialization variance for stable kernel evolution, and stochastic gradient regularization for flat minima discovery. To ensure generalization and stability, we employ a tenfold cross-validation strategy and monitor both the validation set performance and gradient norm for early stopping.